2026

A Systematic Mapping Study on Security Requirements & Vulnerabilities in E-Voting

SUPERVISED BY ELDA PAJA

MAX CHRISTIAN SØRENSEN (MAXS) & AHMED NAJEM KHALAF (AHKH)

Abstract

Context and motivation. E-voting systems have the potential to improve the voting process by making it more efficient and increase voter turnout, however, they still face obstacles to adoption in relation to security and trust.

Problem. Despite security and trust being primary barriers to e-voting adoption, existing studies tend to focus on specific technologies or use predefined security properties, leaving the relationships between technological architectures, security requirements, and vulnerabilities largely unmapped across the last decade of research. **Method.** We conduct a systematic mapping study over e-voting literature from 2015 to 2025 focusing on technological architecture, security requirements, and vulnerabilities discussed in papers, as well as the paper types. **Results.** We find that online systems are frequently discussed, including Helios and Blockchain technologies. The most frequently discussed requirements, like Privacy and integrity, are consistently discussed across all 10 years indicating a consensus that these requirements are crucial for e-voting systems. Attacks discussed are usually coercion and denial of service attacks, which corresponds to attacks described as obstacles to e-voting in practice. We find the field mostly consists of proposals and implementation of systems rather than evaluations, indicating a gap in empirical validation of proposed systems.

Contribution. This study provides an overview of e-voting literature, showcasing developments in technological architectures, security requirements and vulnerabilities discussed in the literature, as well as relations between the 3 dimensions. We provide a set of requirements specifications of the identified security requirements as a practical reference for developers implementing e-voting systems.

1 Introduction

Electronic voting (e-voting) has the potential to make the voting process more efficient and automated [1], increase voter turnout [2], attract younger voters and enabling disabled electors to more easily vote [1].

According to the International Institute for Democracy and Electoral Assistance, out of the 178 registered countries in their 'ICTs in Elections Database', 19% of countries used e-voting either at the sub-national and/or national level, 15% did not use it but feasibility studies were planned/carried out, and 6% of countries had used the technology but later abandoned it citing trust and security of the vote as one of the main reasons [3]. Security and trust in the design of e-voting systems are, therefore, important concerns for the adoption rate of e-voting systems.

Given these adoption challenges, understanding how existing research addresses security and trust becomes essential. To this end, we conduct a Systematic Mapping Study (SMS), aiming to provide an overview of e-voting literature. SMS is appropriate for identifying research gaps and showing how the field has matured over time by classifying e.g. research paper types [4].
We looked at 82 papers from January 2015 to December 2025, picking up where the most recent comparable SMS left off in mapping requirements specifications [5] while capturing the emergence of technologies such as blockchain-based e-voting systems, see section 3.3 for more. We map the papers across four dimensions; the types of technological systems studied, the security requirements discussed, the vulnerabilities reported, and the types of articles in the literature.

More specifically we set out to answer the following research questions:

RQ1: What types of technological architectures are studied in e-voting literature?

- SRQ1: How has this developed over time?

RQ2: What security requirements are discussed in the e-voting literature from 2015 to 2025?

- SRQ1: How has this developed over time?
- SRQ2: What requirements are associated with specific technological architectures?

- SRQ3: What functional and non-functional requirement specifications can be inferred from the security requirements identified?

RQ3: What common vulnerabilities affect e-voting have been reported in literature published between 2015 to 2025?

- SRQ1: How has this developed over time?
- SRQ2: What vulnerabilities are mentioned together with specific technological architectures?
- SRQ3: What security requirements are mentioned together with specific vulnerabilities?

RQ4: What types of research papers are represented in the e-voting literature?

- SRQ1: How has this developed over time?

The technological architecture studied in e-voting literature is important to understand as it serves as an indicator of where research attention is focused i.e. fully online systems or hybrid approaches, as a means of tracking how the field evolves in terms of trends related to specific technologies like blockchain (RQ1).

Given that security and trust are primary barriers to adoption, we need to understand what security properties the literature emphasizes (RQ2) including how they relate to technological systems, as they each have unique properties that may emphasize different security requirements (RQ2 – SRQ2). During our study we found that most of them were not rigorously expressed as actionable software requirements, motivating us to create a list of functional and non-functional requirements that can serve as practical reference for developers implementing secure e-voting systems (RQ2 – SRQ3).

Since vulnerabilities represent the concrete risks that undermine security and trust, we identify discussed vulnerabilities in the literature (RQ3) and map their relationships to both specific technologies (RQ3 – SRQ2) and security requirements (RQ3 – SRQ3), which is essential for identifying patterns related to vulnerabilities and different types of systems, and what threats are seen as broadly relevant across system properties.

Understanding what kinds of studies make up the literature reveals potential gaps in relation to specific types of studies over/understudied in the literature (RQ4).

For all research questions, we provide a view over the 10-year period, to showcase how the field has developed over time (SRQ1 for all RQ), as it can identify whether a finding is consistent across years or part of a shorter trend.

2 Related literature

Several systematic literature reviews like [6], [7], [2], [8], [9], [10] and systematic mapping studies [11], [5] have been done on the topic of e-voting. Some focus primarily on reviewing literature on blockchain solutions like [2], [10], [8], [11]. relation to RQ1, we noted down all technological architectures mentioned, not just a specific technology like blockchain making our study broader.

There are also broader studies that have analyzed multiple technical solutions for e-voting, like Barelli et al. [7]. In their study, they identify properties related to e-voting systems based on previous work like privacy and integrity, and the recommendations from the Council of Europe [12]. They also study on-site vs remote technologies as well as blockchain systems studied in the literature and analyses attacks done on employed systems [7]. In relation to our study, we use iterative coding to identify security requirements related to e-voting instead of using predefined properties to measure how often they are discussed, see section 4.4. This way we can analyze what kinds of properties are discussed and if they consistent over time. We also identify attacks discussed in the literature and not only attacks deployed on real systems, highlighting more theoretical attacks. This also allows us to compare what attacks the literature discusses to what attacks are used on employed systems and if there are any notable gaps, see section 6. Additionally, we develop a set of functional and non-functional requirements based on the security requirements identified, providing a set of specifications that can serve as practical reference for developers implementing secure e-voting systems.

Other studies have also focused on requirement specifications, like the one from Sepulveda et al. [5], an SMS focused on non-functional requirements. They looked at four non-functional requirements; security, performance efficiency, reliability and operability. We look at security

requirements with similar overlap, although we use a more fine-grained coding scheme providing a more detailed overview of security requirements in the literature.

Collectively, these studies leave the relationship between specific technological architectures, security requirements, and vulnerabilities largely unmapped, which is the gap our study addresses.

3 Scope

In the following sections, we go over the scope of our research project, key terms and definitions, and a description of an initial search of the e-voting literature we did to define our scope.

3.1 Initial search

We conducted an initial search before our systematic search described in section 4.3 to 1) get a understanding of the field and define our inclusion and exclusion criteria 2) find seminal papers for snowballing, and 3) analyze keywords from relevant texts to develop a search string for the systematic search to find relevant papers.

We started by using broad search strings like “e-voting OR i-voting". We then added papers that we deemed relevant to our research questions in an excel ark with metadata including keywords, abstract, findings and number of citations. We then snowballed from those papers to find more relevant papers, noting down their metadata. During this process we read the title, abstract, and conclusion to determine whether it was relevant.

After gathering 40 relevant papers, keywords began to converge towards the same terms. We then stopped the search to find snowballing candidates, developed our search string based on the keywords, and set the temporal scope.

During this process, we chose one text for snowballing by Bernhard et al. [13] since it had a strong focus on security requirements in e-voting and was heavily cited.

3.2 Key concepts and definitions

We use the following definition for **e-voting**: a voting process enabling voters to securely cast a ballot over a network [1, p. 539]. We also include similar terms, such as internet voting.

We use **‘technological architectures’** to refer to the technical systems underpinning e-voting

systems, such as blockchain technology, DRE systems that enable hybrid voting systems or specific e-voting platforms like Helios. We include both fully online and hybrid e-voting systems. We define a hybrid system as ballot voting at a polling station enhanced by technology in some way e.g. using a tallying machine.

We define a **security requirement** as a property that an e-voting system can have that achieves a specific security goal.

We define **vulnerabilities** as weaknesses or attacks in e-voting systems that could be used to compromise the security or outcome of an election. This includes both theoretical attacks mentioned as relevant in the literature and real exploits found in implemented systems.

3.3 Inclusion and exclusion criteria:

In this section we go over some of the important inclusion and exclusion criteria in our SMS, see table 1 for all details.

Table 1: Inclusion & Exclusion criteria

Inclusion criteria	Exclusion criteria
Studies was written in English	Publications written in any other language than English.
Publications should be accessible through authors institution	Publications that weren't accessible (requiring payment)
The studies should have been published between the years 2015 - 2025	Studies before 2015 and after 2025
The manuscripts must be of international journals or conferences	Magazines, newspapers, blogs
Focus on security architectures and e-voting protocols	Studies focusing solely on technological efficiency or cryptographic applications without analyzing its use in relation to e-voting
The studies have been peer-reviewed	Grey literature that has not been peer-reviewed

We decided to limit our search to papers from January 2015 to December 2025. We chose to capture data from 2015 to include early research into now prominent technologies in e-voting literature such as blockchain based e-voting systems [15]. And as far as the authors are aware based on searching through Google Scholar, ACM Digital Library, Semantic Scholar and Springer Nature Link, the most recent SMS on non-functional requirements in e-voting literature

was published May 2015 by [5], meaning we capture the decade after its publication. The end date December 2025 was chosen to set a clear start and end date to make reproducibility easier.

We excluded grey literature because it has not been peer-reviewed, questioning the validity of the results. Accessibility to articles was important; we only included articles that we had access to through our institution, that were freely available, or where a legally distributed version of the text was simple to find without having to purchase access. This inherently limits what databases we could use and what articles we could access, see section 7 for a discussion on this.

We only included articles with a significant focus on e-voting and relevance to security. Papers that write about e-voting without relating it to security e.g. a paper about blockchain based e-voting only focusing on efficiency, is not included in the scope e.g. [16]. Papers focusing on a specific topic within security that are not specifically about e-voting e.g. a paper on a cryptographic primitive, are excluded if they only mention e-voting as a potential use case without detailing how. If they go into detail on how it applies to e-voting they are included, even if they do not exclusively analyze e-voting as a case.

4 SMS

4.1 Search string

For the systematic search, we developed a search string that represented relevant literature in e-voting based on keywords from our initial search:

("electronic voting" OR "e-voting" OR "internet voting") AND ("coercion resistance" OR "verifiability" OR “integrity” OR "vulnerability" OR "attack" OR "usability" OR "human factors")

A paper was included if it mentioned e-voting and security requirements or attacks, the broad criteria was to limit the risk of excluding relevant texts that did write about both vulnerabilities and security requirements, but only used keywords related to one of them. There is an inherent limitation in the search string, if we increased the variety of keywords, we could have found more relevant papers. However, it could also have introduced more irrelevant articles. See section 7 for a discussion on this.

4.2 Reading guidelines

We needed clear guidelines in relation to what degree to read articles when doing the systematic search and snowballing to gather data (pre-processing), as well as for coding the papers once they had been gathered.

When reading an article for pre-processing and coding, we read the title, abstract, and conclusion, including the method and findings section if it was not possible to determine whether it was relevant or not possible to categorize the text during coding. For the coding process specifically, we also used keyword search to more easily search papers for relevant security requirements, and vulnerabilities.

4.3 Search

For our systematic search we chose ACM Digital Library and Semantic Scholar as our primary search engines because it was free to access most of the papers through them. From ACM Digital Library we got 542 results [17] and 68 results from Semantic Scholar [18]. dblp was also used, specifically to find articles from established e-voting proceedings and specialized venues.

When using our exclusion and inclusion criteria, we included 53 papers from ACM Digital Library, and 3 from Semantic Scholar.

We reviewed papers in the e-vote-id proceedings from 2015 to 2025 [19] to find papers that were potentially not included in our chosen search engines or not caught by our search string. Doing this, we found 12 texts that were relevant to our study.

During our snowballing of [13] we found 100 results via Google Scholar, of which 11 articles, not already in our dataset, were chosen as relevant.

The snowballing papers mostly came from search engines we had not used like Spinger Nature Link, although one was available on ACM Digital Library and one was available on Semantic Scholar. None of them were caught by our search string, so the snowballing helped find two papers that would otherwise not have been found in our systematic search. See a discussion on this in section 7.

In total, we gathered 89 papers, 6 of them were duplicates which we removed, and also 1 that was deemed not related to e-voting after discussion, leaving 82 papers in our dataset.

Before coding, we first developed keywords to more easily find security requirements in the body of the articles as they were rarely mentioned in the title, abstract, and conclusion of the papers. To develop keywords for security requirements, as well as vulnerabilities, we analyzed 10 of our gathered papers in detail to generate keywords, while also doing preliminary coding of those texts. See section 4.4 for more on this process.

The papers were chosen mainly based on a quick assessment of how much they dealt with security requirements by reading the title and abstract. We picked 8 others using the same method, and chose [13] as one of the articles since we knew that security requirements were a focus of the paper.

To develop the keywords, we noted relevant words used for described security requirements. Keywords began to converge after we had analyzed 5 texts and after the 7th article no new relevant keywords were found. This indicated that our keywords were likely representative for most of our dataset, so we stopped after analyzing the 10 articles we originally decided on.

From there we developed two regex-based patterns with mostly partial matches and were case insensitive to look through the articles.

Pattern 1 - vulnerabilities:

"attack|vulnerability|exploit|payload|compromise|forge|tamper|adversary|malware|threat actor|mistake|fail"

It is mainly compromised of general words to find vulnerabilities such as “attack” which would match with e.g. “ballot replay attack” and “clash attack”.

Pattern 2 – requirements:

"End-to-end Verifiability|Availability|Effectiveness|efficiency|Confidentiality|Receipt-freeness|Software independence|Coercion accountability|uncoercibility|Dispute resolution|verification|Everlasting privacy|privacy|Coercion resistance|coercion-resistance|uncoercionability|auditing|usability|practicality|correctness|eligibility|unreusability|fairness|Election fairness|mobility|Vote-and-go|Safe

```
aggregation|authentication|Equal voting rights|integrity|Vote
correctness|robustness|scalability|Long-term privacy|Cast-as-Intended
verifiability|Recorded-as-Cast verifiability|Counted-as-
Recorded|verifiability|anonymity|Verifiability|reliability|secrecy|accountability|\blaw\b|
Local law|State law|National law|legal|requirement|committee"
```

This regex mixes specific requirements identified with more general words like "requirement" which can catch some requirements outside those identified.

4.4 Classification scheme

To answer our research questions, we wanted to categorize papers based on 3 dimensions: what type of security requirements are discussed in the paper, what type of vulnerabilities are discussed in the paper, and what type of paper it is. We also tagged technological architectures analyzed in the papers, and when one was mentioned in less detail.

We developed 3 classification schemes with the aim of a set of categories that represent the body of gathered papers [4, p.4].

To develop our codes, we followed an iterative coding process similar to the one described by Lichtman:

1. As mentioned in the reading guidelines section, we read the title, abstract and conclusion, of each of the 82 papers, including keyword search, noting down themes that emerged in relation to the type of paper, security requirements and vulnerabilities.
2. We then revisited our initial coding aiming to collapse irrelevant ideas, rename codes with inconsistent names, and potentially revise codes.
3. We then categorized codes, most of them fit neatly into distinct categories whereas others fit better as subcategories for more abstract themes.
4. We then considered the current categories, e.g. whether some should be changed, merged due to similarity, or removed.

[20, p. 252]

While we have enumerated the steps they are not linear e.g. "tamper attack (hardware)" used to be a code until we decided it could fit better under the code "hardware exploitation" instead.

After having developed the coding scheme, we each coded all papers independently, and then discussed any discrepancies between the results, a procedure commonly used in systematic review practice [21].
Discrepancies were primarily due to missed instances of codes, not different interpretations of what code to use. Our coding schemes largely reflected the definitions and wordings used for vulnerabilities and security requirements in the literature, meaning few of the discrepancies were due to interpretation errors. E.g. the code "clash attack" is used for cases of clash attacks, which is widely cited using that wording in the literature. Security requirements are similar, most of the time it is easy to identify when a security requirement is referred to, e.g. long-lasting or everlasting privacy is usually mentioned by one of the two accepted terms for it.
Even with keyword searching to support the manual reading, it was possible to miss cases, so our approach aimed at maximizing coverage when mapping the literature, which is why we used double coding. This approach can help reduce data extraction errors [22]. However, reproducibility at the level of identifying every instance cannot be guaranteed, as detection depends on careful reading and may vary between reviewers. Although the detailed coding schemes should support consistent identification and classification of codes.

Note that we do not think the categories in the three coding schemes are exhaustive, it is possible to find more categories or to develop a different coding scheme to classify the literature e.g. a less detailed coding scheme. We chose to develop the scheme towards giving a more detailed overview of the literature, as a finer level of abstraction allows for a more precise mapping of research gaps and a clearer understanding of how specific technical threats interact with security properties.

Below is an explanation of how we developed each coding scheme and how we applied the codes. See Appendix 1, 2, and 3 for details on the actual codes in each coding scheme.

For the **paper type coding scheme** we noted down how the article described itself, see Appendix 1 for an overview of the codes. We tried to categorize the article using their own terminology, not validating whether it fit the type of study it defined itself as. Some of the codes directly reflect the same categories used by Horkoff et al. [23], namely proposal, formalization, meta study, case study and implementation.
An article can have more than one type, for example, an article can do a formalization and based

on this implement an e-voting system. That article would be coded both as “formalization” and “implementation”.

Our **security requirements coding scheme** was developed to identify discussed or implemented security requirements, which can be found in Appendix 2. We utilized keyword search to help identify security requirements while being vigilant that the case fits with our code definitions, by assessing whether the case was about a security requirement when using keywords.
However, it was not always clear if the security requirement was implemented in a voting system or simply discussed as a property it lacks/should have. Even if it was not completely clear it was implemented, we decided to code it. This was in part done to reduce the time it would take to properly analyze the overall context. See section 7 for a discussion on this.

Our **vulnerability coding scheme** was developed to identify vulnerabilities discussed in the literature and can be found in Appendix 3. As mentioned we utilized keyword search to help identify vulnerabilities, while being conscious of what code definition an attack would fall under.

For the security requirements and vulnerability classification schemes we opted for a more detailed coding scheme. This was in part because the literature followed distinct definitions for different attacks and security requirements, e.g. man-in-the-middle attack and coercion-resistance (see Appendix 3 and Appendix 2). There were also some more abstract codes. Privacy acted as an umbrella term for several other definitions that did not differ enough from the general idea of “privacy” to warrant a unique code, like “ballot secrecy”. Although in the cases where the idea was distinct enough, it did receive a unique code like “everlasting privacy” (see Appendix 2).
This is also reflected in the examples, some code examples are simple and highlight that this code applies when a paper clearly states they use that security requirement, see “end-to-end verifiability” in Appendix 2. Those with broader definitions have more varied examples to help, see “privacy” in Appendix 2.

When tagging technologies, we wrote down the name used in the paper and asserted whether the technology was only mentioned briefly or had a greater analytical focus in the study.

5 Analysis of results

In the following sections, we go over the results of our systematic search, answering our research questions.

5.1 RQ1: Technological architectures

In this section, we analyze what types of technological architectures are studied in e-voting literature.

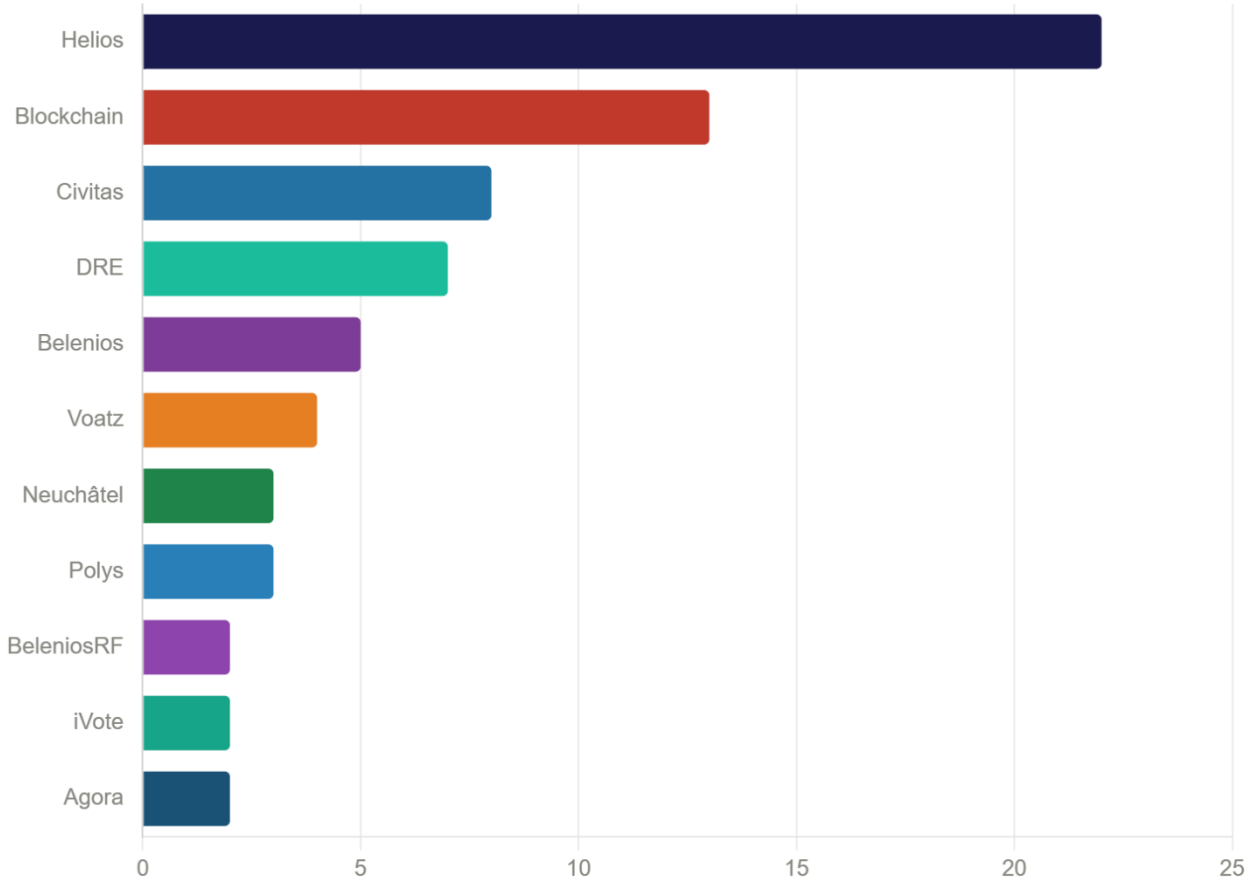

Figure 1 - Most studied technological architectures. The blockchain category includes all blockchain platforms that are not established systems like Agora.

Figure 1 shows the most studied e-voting systems across all 82 papers. Helios, a web-based e-voting system that uses a central server to run elections, is mentioned most often with 22 mentions. However, only 11 of these papers focus on Helios [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], where the remaining 12 [35], [36], [37], [28], [38], [39], [40], [41],

[42], [43], [44], [45] briefly mention it using it e.g. in a literature review or as a comparison point, rather than being studied in isolation only.

Blockchain is mentioned 13 times [15], [46], [47], [48], [49], [50], [51], [52], [53], [54], [55], [56], [57], 15 as a group if we include Polys [58], [59] and Agora [58], [59] which are also blockchain based. This group reflects the interest in blockchain's unique characteristics like immutability of blocks and verifiability through the ledger being distributed and decentralized in multiple locations [60, p. 983]. This is important for e-voting as blockchain can satisfy some of the security requirements discussed in the literature like verifiability and help prevent e.g. tampering.

Civitas and DRE systems rank third and fourth. The presence of DRE systems is worth noting. They represent the interest in hybrid e-voting systems compared to fully online e-voting systems in the literature. All the other voting systems in the figure like Helios, Civitas, Belinios, Voatz and the different blockchain systems, are designed to be fully online and account for most of the mentions in the literature, reflecting that most of the interest in the literature, are on online voting systems as opposed to systems designed for hybrid voting. See section 6.3 for more on this.

Overall, the distribution is heavily focused toward a small number of systems. This concentration may reflect both maturity of certain systems and could imply a tendency to build on well-established work rather than evaluating less documented systems.

5.1.1 SRQ1: Technological architectures over time

In this section, we analyze how this has developed over time.

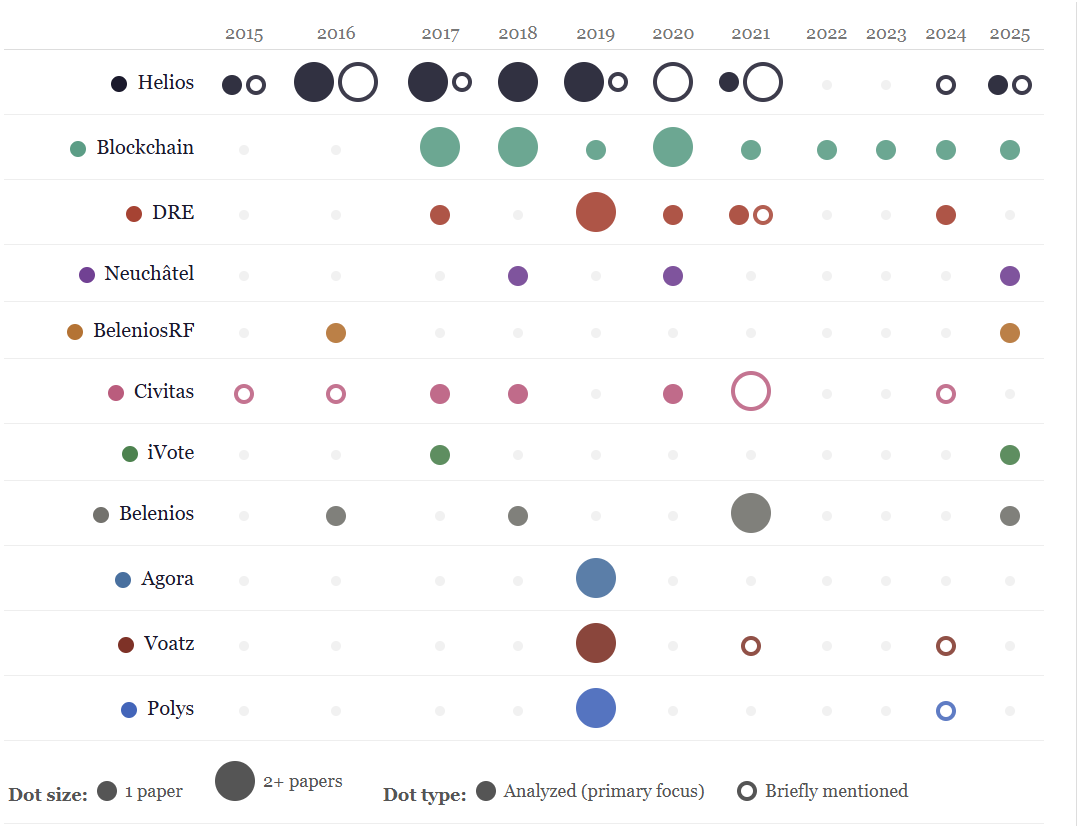

Figure 2 - Technology appearances across the study period (2015–2025). Larger dots indicate 2 or more papers.

As shown in Figure 2, the most studied system in our dataset between 2015 and 2019 was Helios. After 2020, Helios largely disappeared from our dataset, which suggests the research community now sees it as a well understood case rather than something that needs more investigation.

From around 2017, we see a clear shift toward blockchain-based e-voting systems. As shown in Figure 2, blockchain continues to appear in the literature all the way through 2025, showing that it has become an established area of research rather than a short trend.

2025 data shows systems like Belenios, IVote, and Neuchâtel appearing again as part of a meta study on usability in e-voting systems [34].

5.2 RQ2: Security requirements

In this section, we analyze what security requirements are discussed in the e-voting literature from 2015 to 2025.

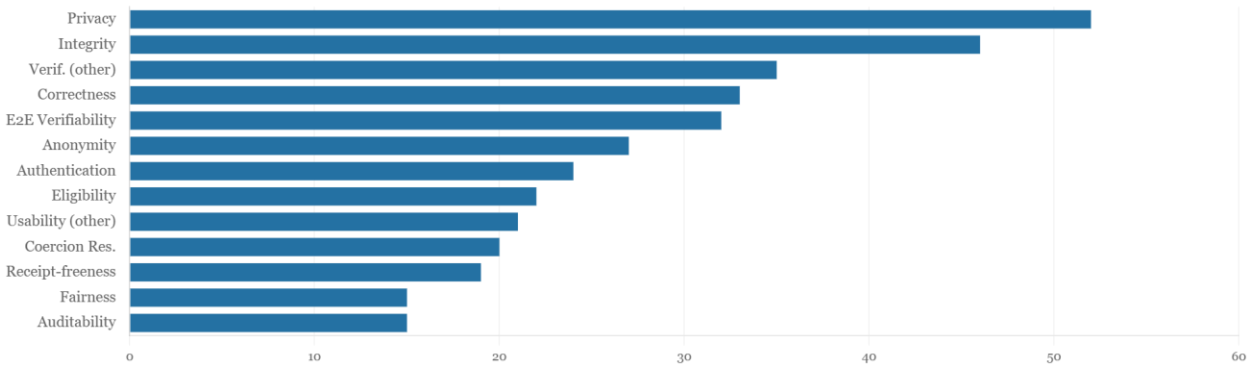

Figure 3 - Top 13 security requirements mentioned across papers.

This section looks at the security requirements most frequently mentioned across our dataset, as shown in Figure 3.

Privacy is the most frequently mentioned requirement, appearing in 52 papers (63% of the dataset), and integrity is also mentioned in over half of the papers with 46 mentions (56% of papers). This is in accordance with other papers placing it as the most critical security requirement in e-voting literature [7].

Verifiability (other), Correctness and end-to-end verifiability each appear in 32-35 papers. This cluster accounts for a large share of the dataset and reflects the field's strong interest in ensuring that election outcomes can be verified and are correctly counted.

Anonymity, appears around 27 times in the data. This could indicate that it is not as large a priority to anonymize votes after an election, compared to keeping the details of the votes themselves secret. Authentication and eligibility each appear in around 24, and 22 papers respectively, reflecting the importance of ensuring that only authorized voters can participate in an election and that their identity can be verified.

Coercion resistance appears in 20 papers, which may reflect the fact that coercion resistance is a complex property to achieve in practice [61].

Fairness and auditability appear in roughly equal numbers, both sitting around 15 papers. Availability has just over 9 mentions, indicating that while system uptime is recognized as a requirement, it receives considerably less attention than other security properties.

We also tracked mentions of legal requirements, where the system was developed for a specific legal context or acknowledged that legal requirements were important for a secure implementation. This was mentioned 7 times, indicating this is not a large concern. It might also relate to the types of papers in the literature; only 2% of papers were case studies, see section 5.4.

5.2.1 SRQ1: Security requirements over time

In this section, we answer how security requirements discussed in the literature have developed over time.

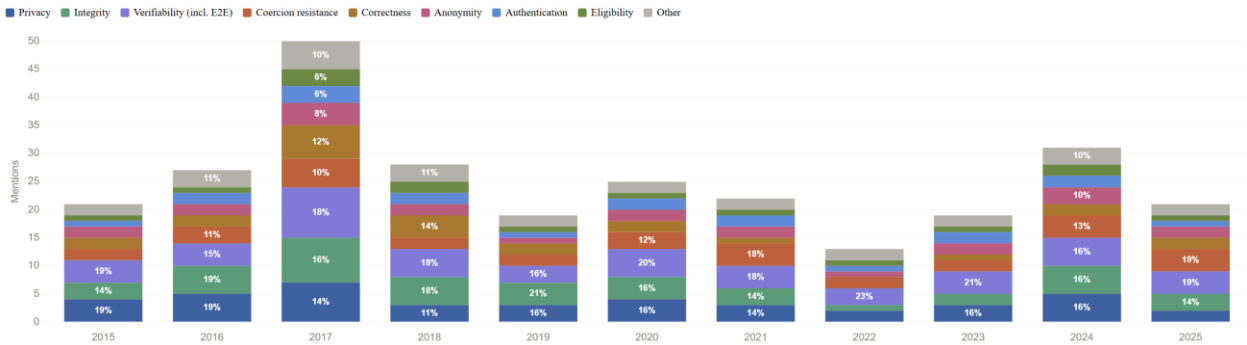

Figure 4 - Security requirement distribution over time (2015–2025). Each segment corresponding to a security requirement shows its relative percentage of the year's total mentions. If a block accounts for less than 10% the percentage is not visible. Other accounts for all other requirements not individually mentioned.

As shown in Figure 4, most of the frequently mentioned security requirements appear consistently throughout each year like privacy. Only correctness did not appear in 2022. This showcases how the most frequently mentioned requirements are consistently mentioned in the literature over time. They are also relatively consistent in relation to how many mentions they account for each year, indicating that these requirements are well-established concepts with consistent relevance to security of e-voting systems. This is supported by the fact that ‘other’, codes accounting for less than 10% of the mentions that year, accounts for at most 11% in the dataset.

5.2.2 SRQ2: Security requirements in relation to technological architectures

In this section, we look at what security requirements are associated with specific technological architectures.

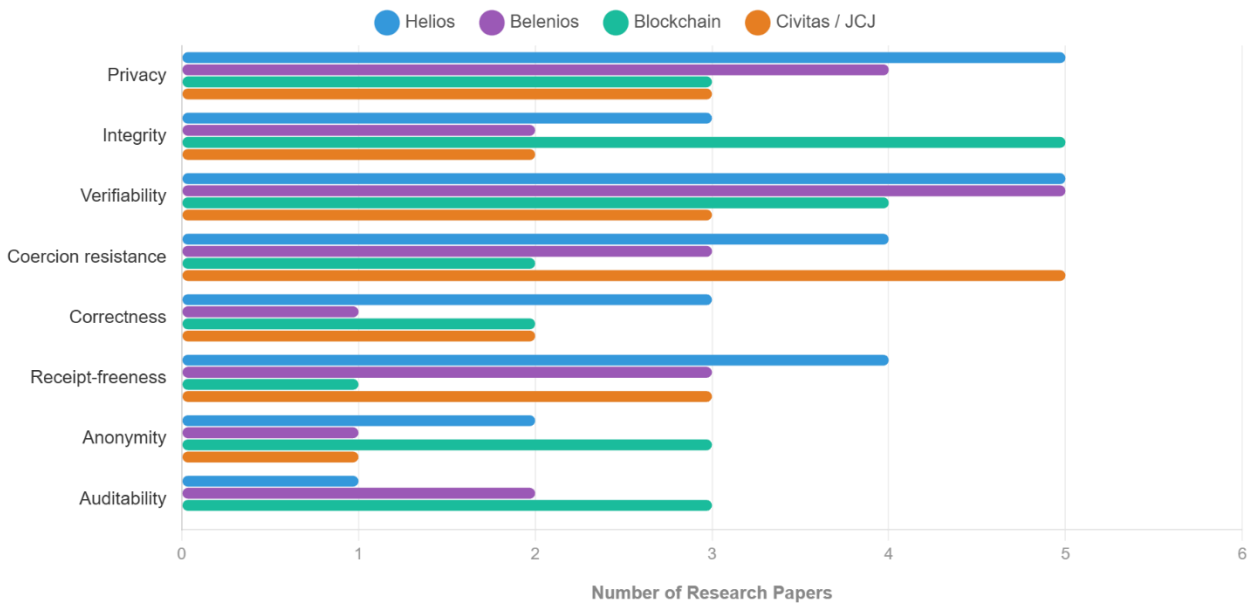

Figure 5 - The most mentioned security requirements in relation to specific technologies. Belenios and BeleniosRF are combined.

Figure 5 shows the most mentioned security requirements that were associated with specific technologies. Some requirements are mentioned more frequently than coercion resistance like eligibility, however it was not associated with as many named e-voting systems or technologies. The figure shows us that privacy, integrity, verifiability, coercion resistance, correctness, receipt-freeness, anonymity, and auditability were frequently mentioned together with specific e-voting systems. This could highlight that these security requirements are especially important when discussing implementations of e-voting systems. This reflects the literature where privacy and integrity are seen as core properties for e-voting systems [7], and concerns in practice where coercion resistance is an important property specifically for online voting systems [14]. However, some requirements like availability are not mentioned as often which is otherwise seen as important, especially in relation to employment of systems, see section 6 for a discussion on this.

Additionally we see that privacy is discussed frequently for all systems, however, particularly in relation to the Helios system. Integrity is mentioned often in relation to blockchain systems, perhaps because Blockchain technology's immutability can help support this security requirement. Other systems like Civitas are built with coercion resistance in mind, explaining why the security requirement is mentioned often with Civitas [27].

5.2.3 SRQ3: Requirements specification based on security requirements

In this section, we develop functional and non-functional requirement specifications (hereby referred to as FR and NFR) based on the security requirements in our coding scheme.
We provide 1-2 functional and non-functional requirements that relate to each security requirement, except for legal requirement which was used more for tracking how often papers mentioned how legal requirements were important in general/to their system.
The examples are purely hypothetical; some are designed for online systems, whereas others only make sense in hybrid systems. We show the FR and NFR for the 8 most stable security requirements identified in section 5.2.1 as they are more likely to be relevant to most systems. We have also developed FR and NFR for the other security requirements in our coding scheme, they are in Appendix 4 for those who are interested in a specific requirement.

End-to-end verifiability:

- **FR1:** The system shall provide each voter with a unique tracking code after casting their vote, which they can use to verify that their vote appears correctly in the published tally.
- **FR2:** The system shall publish a cryptographically signed bulletin board containing all cast ballots and the final tally, accessible to any third-party auditor.
- **NFR1:** The system's verifiability mechanisms shall function correctly regardless of the underlying software or hardware used to conduct the election, such that no undetected software error can produce an undetectable change in the verified outcome.

Verifiability (Other)

- **FR1:** The system shall ensure that the list of participating voters is publicly verifiable, allowing any observer to confirm the total number of votes cast without revealing the identity of individual voters.
- **NFR1:** All verifiability artifacts shall be archived and publicly accessible for a minimum of five years following the election.

Correctness

- **FR1:** The tallying module shall apply a verifiable homomorphic computation or mix-net process to ensure all valid ballots are counted exactly once and no invalid ballots are included.
- **NFR1:** The system shall produce a correctness proof verifiable by an independent auditor within 2 hours of the polls closing.

Privacy

- **FR1:** The system shall encrypt all ballots at the point of entry using public-key encryption, such that no single authority can decrypt an individual ballot.
- **FR2:** Voter identity and ballot content shall be stored in separate, access-controlled databases with no way to query a join between them.
- **NFR1:** Privacy protections shall remain effective against a passive adversary observing all network traffic throughout and after the election.

Eligibility

- **FR1:** The system shall verify each voter's eligibility against the official electoral register before granting access to the ballot.
- **FR2:** In hybrid deployments, eligibility checks performed at physical polling stations shall be synchronized in real time with the central electoral register to prevent duplicate participation across channels.
- **NFR1:** The system shall complete eligibility verification within a response time that does not meaningfully delay the voter's access to the ballot, even when a large number of voters are authenticating simultaneously.

Authentication

- **FR1:** The system shall authenticate each voter using multi-factor authentication before granting access to the ballot.
- **NFR1:** Authentication credentials shall be invalidated immediately after use and shall not be reusable within the same election period.

Integrity

- **FR1:** All data transmitted between the voter's device and the voting server shall be protected by end-to-end encryption, preventing any intermediary from altering ballot content in transit.
- **NFR1:** Any detected tampering with stored ballot data shall trigger an automatic system halt.

Anonymity

- **FR1:** The system shall ensure that the link between a voter's identity and their ballot is permanently and irrevocably broken before the ballot is included in the tally.
- **NFR1:** No combination of system logs, network captures, or database records shall be sufficient to link a specific unsealed ballot to a specific voter.

5.3 RQ3: Vulnerabilities

In this section we analyze what common vulnerabilities affect e-voting have been reported in literature published between 2015 to 2025.

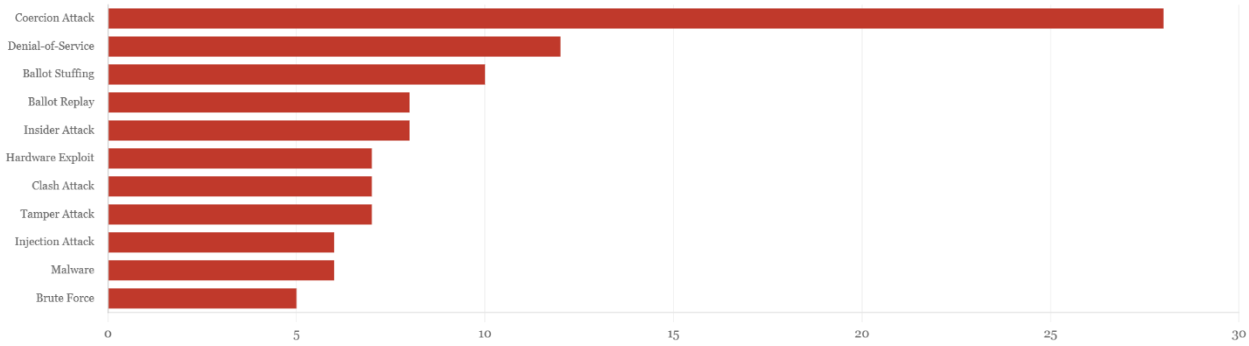

Figure 6. 11 most frequently mentioned vulnerabilities in the data. We chose to highlight vulnerabilities that were mentioned at least 5 times.

The most significant finding in Figure 6 is the high mentions of coercion attacks, which appear in 28 papers. This reflects the central challenge of remote e-voting: while cryptographic protocols can effectively secure the digital ballot box, they struggle to protect a voter who is being intimidated or bribed in an uncontrolled environment outside of a polling station [62], [63]. The high frequency of coercion attacks in the literature suggests that researchers view voter coercion in a remote setting as one of the most critical threats to address in e-voting system. This

is further reflected in the number of papers proposing dedicated coercion-resistant systems [61], [64].

Denial-of-service (DoS) is the second most frequently mentioned vulnerability with 12 papers, and ballot stuffing follows closely with 10. DoS can effect even cryptographically secure systems by taking them offline during an election, excluding voters regardless of protocol strength [53]. This highlights why this is an important issue in the literature as it is a threat that cannot be counteracted simply by cryptographic secureness and can be difficult to defend against.

Ballot replay, insider attack, hardware exploitation, and clash attack each appear in 7 to 8 papers. Hardware exploitation, while appearing less frequently, represents a real world threat as demonstrated by analyses of physical voting machines such as DRE systems [65]. This highlights an important distinction in the data, some vulnerabilities are primarily discussed as theoretical threat models, while others have been demonstrated in election systems.

Brute force appears 5 times. In the context of e-voting, this suggests that modern cryptographic primitives such as elliptic curve cryptography and lattice-based schemes [66, p. 346] make brute force attacks infeasible for current adversaries. In relation to this, since 2019, 3 articles have mentioned quantum attacks, like quantum-enabled brute-forcing. This shows that while quantum attacks have gained awareness as a potential issue, it has not gained a lot of traction in the e-voting literature.

5.3.1 SRQ1: Vulnerabilities over time

In this section, we analyze how vulnerabilities discussed in the literature have developed over time.

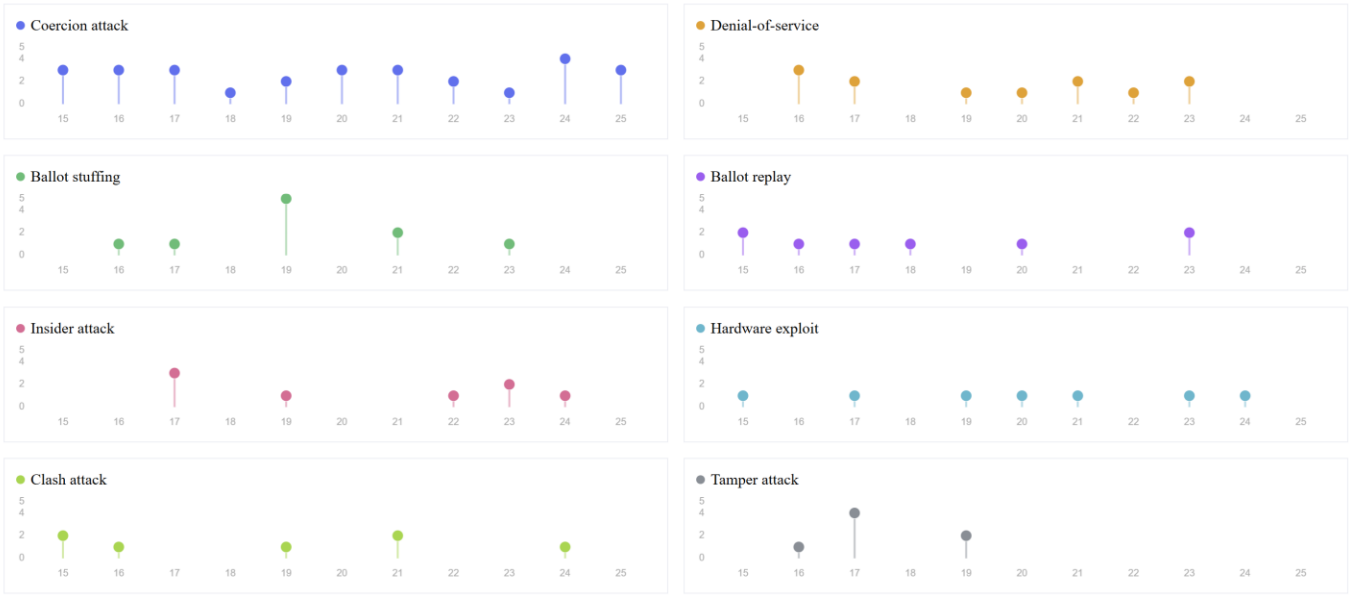

Figure 7 - Vulnerability mentions per year, one panel per vulnerability type (2015–2025). Shows the 8 most mentioned vulnerabilities

As shown in figure 7, coercion attacks are mentioned at least once every year over the last decade, only with small dips, highlighting how coercion attacks are consistently seen as prominent threats in the literature throughout the last decade.

DoS attacks are also mentioned frequently showcasing how the attack is consistently relevant in the literature, even if it is not mentioned every year.

Other attacks are more infrequent, such as ballot stuffing which peaks in mentions in 2019 with 5 mentions. However, it is barely mentioned in both prior and later years, totaling 5 mentions if we ignore 2019. The peak could indicate a brief interest in the topic, however, one that also quickly faded away.

Ballot replay, insider, and clash attacks as well as hardware exploitations have gaps between some years but are mentioned at least half of the years showcasing some consistency, while not being frequent topics. Tamper attacks and other attacks not highlighted are more sporadic or only occur during a short period of time. For example, tamper attacks were only mentioned between 2016 and 2018. This could be due to lack of interest over time like with tamper attacks, or a surge of interest in the later years like quantum attacks mentioned first in 2019.

Other attacks are only mentioned once or twice which could be due to a lack of relevance to the broader study of e-voting.

5.3.2 SRQ2: Vulnerabilities in relation to technological architectures

In this section, we analyze what vulnerabilities are mentioned together with specific technological architectures.

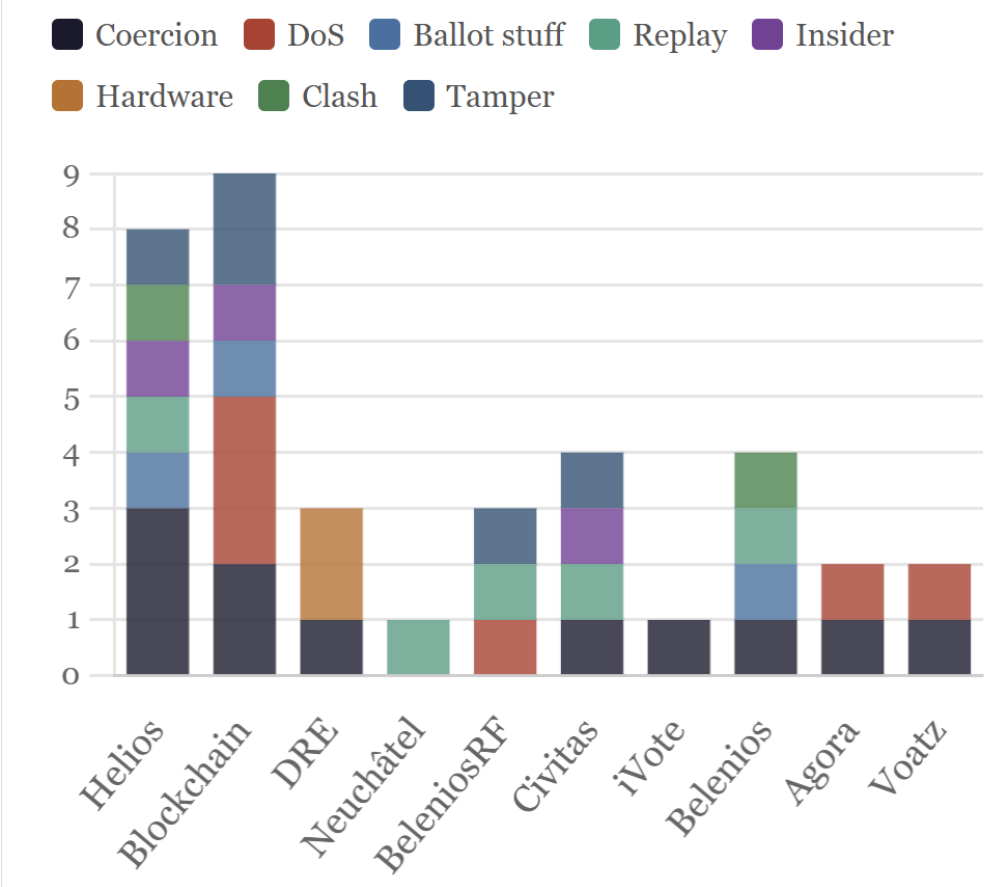

Figure 8 – Stacked bar chart of vulnerability types per studied technology, each colored block corresponds to a unique vulnerability, the y-axis shows how many vulnerabilities are mentioned, while the x-axis corresponds to different e-voting systems/technologies used for e-voting.

As showcased in Figure 8, blockchain-based e-voting systems as a group were frequently mentioned together with DoS attacks as well as coercion attacks. DoS attacks are likely discussed attacks and not seen as a threat to blockchain, as blockchain is decentralized and can function even if some of the nodes become offline as they can be synced with other nodes when brought back online [10]. In relation to coercion, Abuidris et al. highlights how it is an issue for

blockchain systems as e.g. vote-buying is highly scalable [58, p.100]. Similarly, all systems, except Neuchâtel and BeleniosRF, are commonly mentioned together with coercion attacks, highlighting how this issue applies to several online systems. BeleniosRF focuses heavily on limiting coercion [36] which is likely why coercion attacks are not mentioned as an issue in relation to the system.

Helios is related to almost all of the 8 most mentioned vulnerabilities, including less mentioned vulnerabilities like injection attacks not included in the figure [25]. This could be a reflection of how often Helios is studied in the literature. Coercion attacks are often mentioned with Helios, since it is made for small scale online elections it makes sense when discussing its use in bigger elections where coercion might be an issue.

For DRE technology used for e-voting systems, hardware exploitation was mentioned twice with the technology, which makes sense since given that DRE systems are physically available at a polling location and can, therefore, be the target for hardware exploits. Fully online systems do not have the same concern unless they have a physical channel they connect with which can explain why the other systems in Figure 8 are not mentioned with hardware exploitation.

5.3.3 SRQ3: Vulnerabilities in relation to security requirements

In this section, we analyze what security requirements are mentioned together with specific vulnerabilities.

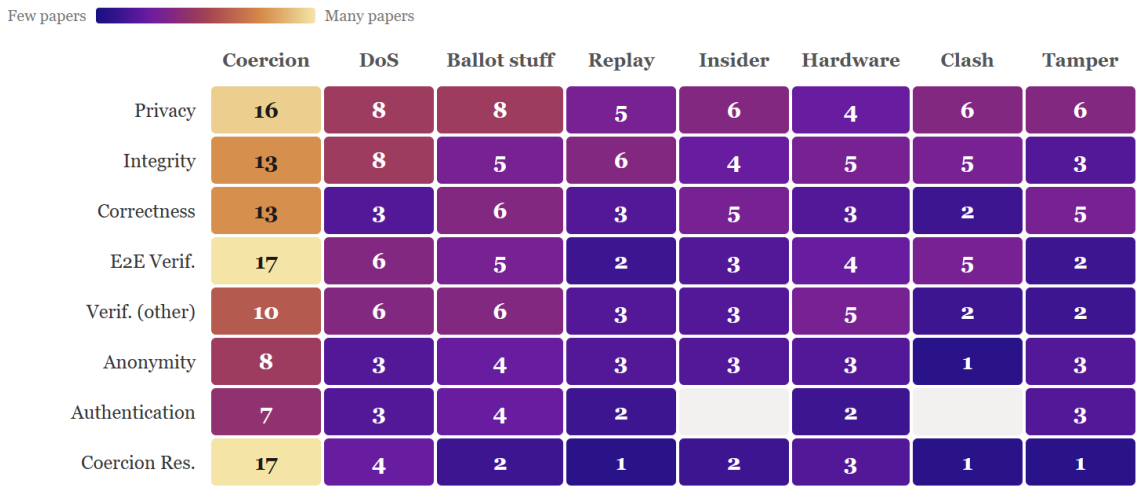

Figure 9. Co-occurence heatmap – requirements (rows) x vulnerabilities (columns), cell value = number of papers.

In Figure 9 we see that coercion has the highest co-occurrence across all major requirements. The two largest pairs in the entire dataset are between coercion attack and end-to-end

verifiability (17 papers) and coercion resistance (17 papers). This concentration reflects a central tension in e-voting research between verifying the vote without giving the coercer an opportunity to confirm how their victim voted [61], [62]. The high scores across privacy, integrity, correctness, and end-to-end verifiability all intersecting with coercion confirm that coercion attacks are treated as the primary adversarial concern in the literature.

DoS shows a consistent relationship with integrity (8 papers each), acknowledging that a system that is taken offline during an election is effectively compromised regardless of its cryptographic properties [53].

Figure 9 also shows a large area with little to no data. Insider attacks, hardware exploitation, replay attacks, and tamper attacks all show low co-occurrence with requirements, rarely exceeding 5 papers for any single pair. This could indicate that the literature has focused on theoretically well-defined threats such as coercion, while the complex interactions between low-level operational vulnerabilities and core system requirements remain more unexplored [67], [68]. This is also reflected in the paper types, as only 10% of papers are security analyses where low-level vulnerabilities are usually identified, see section 5.4.

5.4 RQ4: Paper types

In this section we answer what types of research papers are represented in the e-voting security literature between 2015 and 2025.

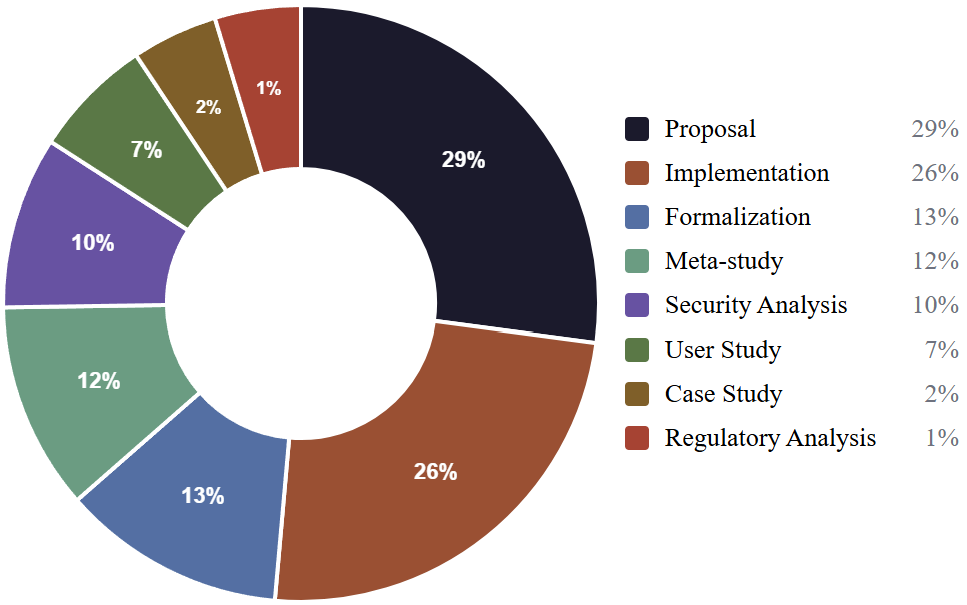

Figure 10 - Distribution of paper types (N=82). Percentages reflect share of total type mentions, as papers can have more than one type.

As seen in Figure 10, the distribution shows a close split between proposal [69], [35], [70], [37], [71], [72], [73], [74], [75], [48], [76], [49], [77], [40], [51], [41], [52], [78], [62], [33], [42], [79], [80], [64], [81], [82], [83], [61], [68], [57], [45] and implementation papers [84], [85], [36], [86], [15], [73], [75], [46], [47], [87], [48], [88], [89], [77], [39], [90], [32], [50], [52], [62], [54], [42], [80], [83], [61], [68], [57]. The high share of proposals indicates that much of the research during this period has been focused on designing and suggesting new systems, protocols, or methods. The 26% share of implementation papers suggests that alongside new ideas, there is also interest in contributing with practical solutions.

The remaining papers are spread across formalization (13%) [24], [91], [72], [28], [29], [76], [92], [93], [62], [33], [42], [79], [61], meta study (12%) [27], [13], [58], [59], [94], [53], [55], [95], [96], [56], [47], [34], security analysis (10%) [26], [97], [98], [31], [67], [99], [93], [43], [100], [68], user study (7%) [30], [101], [41], [102], [44], [34], [63], case study (2%) [25], [103], [38], [47], [104], and regulatory analysis (1%) [105]. The notable share of formalization papers reflects the mathematical and cryptographic nature of e-voting research, where formal verification methods are commonly used to prove security properties [62].

Taken together, the dominance of proposals and implementations points to a field that is still primarily focusing on building and proposing rather than evaluating. There is a group of papers analyzing systems, either in the form of a security analysis (10%), user study (7%), or case study (2%), accounting for around 19% of the combined paper types. It suggests that real-world validation/testing of proposed systems is of interest; however, it still accounts for a small proportion of papers. Moving the field toward more empirical and evaluation-based research would strengthen the practical relevance of existing proposals [106].

5.4.1 SRQ1: Paper types over time

In this section, we answer how paper types have developed over time.

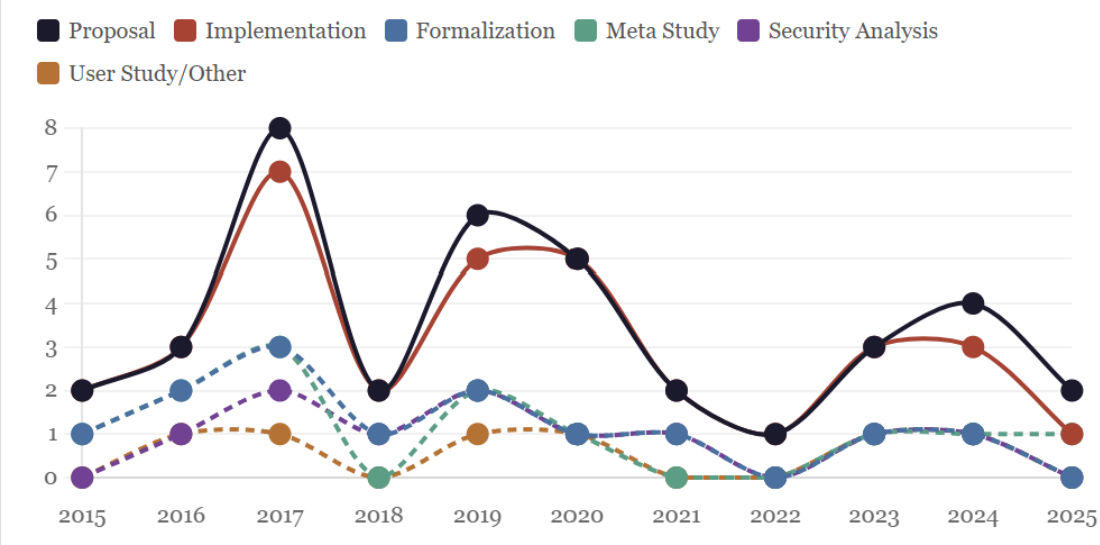

Figure 11 - Paper type distribution over time (2015-2025).

Figure 11 shows how the distribution of paper types has changed across the last decade, spanning from 2015-2025. Several clear patterns appear when looking at the data over time.

2017 stands out as the most active year in the dataset, with 8 proposal papers and 7 implementation papers published in that single year. This aligns with the broader rise of blockchain-based e-voting research during this period [15], [46]. Proposals dominate every single year in the study period, this is particularly visible in the most recent years of the study period, where proposals ranged from quantum-resistant voting protocols [82] to coercion-resistant voting systems [61].

Implementations are the second most frequent appearing in every year, suggesting that practical contributions have remained a stable priority.

Formalization papers appear almost every year of the study period, maintaining a steady presence of at least one paper per year.

Security analysis papers are largely absent before 2017 but increase steadily from that point onward. This might reflect a broader trend in the field toward evaluating the security of existing proposals rather than only proposing new ones. User studies and case studies are more irregular, appearing roughly once every one to two years with long gaps in between. This uneven pattern reflects the difficulty of conducting real-world evaluations of e-voting systems, which require access to live elections or institutional cooperation [97]. While security studies are more consistent after 2017, the low and irregular presence of user and case studies reinforces the finding from section 5.4 that real-world validation remains a gap in the literature.

6 Discussion

In this section, we discuss parts of the analysis and compare our findings about technologies and vulnerabilities from the literature to what types of e-voting systems are used in practice and what kinds of vulnerabilities are seen as important.

Our analysis shows that the e-voting literature between 2015 and 2025 is heavily concentrated on a narrow set of threats, mainly coercion attacks.

The National Academies report “Securing the Vote: Protecting American Democracy” describes malware as one of the greatest threats to electronic voting, noting that it can be introduced at any point in the election and is not easily detected [107, p.87]. In our dataset, malware appears in around 5 papers, making it one of the least discussed vulnerabilities relative to its practical severity. The same report also highlights DoS attacks as a serious threat, noting that even when targeted at a limited number of jurisdictions, DoS can undermine confidence in election integrity and potentially alter an election’s outcome (Ibid., p.86). While DoS is the second most mentioned vulnerability in our data at 12 papers, it still receives far less attention than coercion.

In relation to this, the requirement of availability is only mentioned 9 times even though it is crucial to ensuring defense against e.g. DoS attacks.

Beyond these specific threats, the report also points to a broader issue namely vulnerabilities arise not only from technical design flaws, but from the complexity of modern IT systems and human fallibility in making judgments about safe and unsafe actions (Ibid., p.89). In relation to this, social engineering and insider attacks appear in around 7-8 papers each, despite being recognized as serious real world threats.

As shown in RQ4, security analysis papers and case studies together account for only 12% of our dataset, which may partly explain this gap in relation to DoS and malware. Broader engagement with evaluating systems could help ground future proposals in the kinds of risks that deployed systems face.

Other reports like the European Commission's assessment of e-voting, state that coercion is one of the biggest obstacles to online voting [14]. While there might be a gap between practice and the literature in relation to operational threats that are harder to formalize, such as malware, insider access, and DoS, coercion attack is seen as an important issue for online voting in both practice and in the literature. However, this is only when we take online voting into account, for hybrid systems or traditional ballot voting, coercion is not seen as a prominent threat (Ibid.).

Our analysis shows that most papers focus on online e-voting systems compared to hybrid systems enabled by electronic systems like DRE machines. While electoral voting is not the only place that e-voting systems are used, it can be beneficial to compare interest in online voting systems with practice in electoral elections.
According to the ACE Electoral Knowledge Network, 1% of voters cast their vote via the internet in elections globally [108]. In comparison, 10% of votes were cast via electronic voting machines.

One reason for this discrepancy is that while there might be an interest in online voting, it has issues important to election officials. According to an EU report in 2021 [14], online voting has a large issue in the form of coercion. While it is possible for officials to make sure a voter has casted their ballot without intimidation, it is a bigger problem with online voting. In contrast, the risks associated with electronic voting machines are not seen as high, as they do not have the

issue of voter intimidation. What is an issue for both hybrid and online voting systems is the perceived effect on the integrity and transparency of the election (Ibid.). Low experience with e-voting, emotions and perception of their effectiveness, satisfaction with use and more all have a large effect on adoption of e-voting systems (Ibid.). Usability (other) is mentioned around 21 times which does indicate that the literature is trying to solve this problem.

The discrepancy is not necessarily due to a difference in interest in online voting between academia and election officials, online voting has more obstacles so that is where research is focused. Our analysis supports this claim as coercion attacks were mentioned frequently in the literature, suggesting that researchers view coercion as a critical threat to e-voting systems. Additionally, around 20 papers mentioned coercion resistance as a security requirement, and 19 papers mentioned receipt-freeness indicating that these security requirements are frequently studied.

7 Threats to validity

In this section, we go over the threats to validity we could identify in our study, as per definitions found in [106].

There is an inherent limitation of the temporal scope in that we exclude the most recent literature from 2026 which could be relevant for our study as well as all literature before 2015. This means there is a threat to the *external validity* of our results in terms of how generalizable our work is to studies before 2015 and after 2025. Additionally, we only included papers we could access for free, so while our results should ideally be generalizable to these types of papers, we do not have any data that indicates it is the same for other paywalled papers.

We only snowballed using one paper [13], creating a bias towards that papers reference network threatening the *internal validity*. The systematic search could limit this bias as it did not favor number of citations or the reference network of the papers it found.

In relation to *reliability*, our coding depended on careful reading to identify all the codes in an article, while keyword helped support the manual reading process on this, it is possible that other coders would not find every instance of the codes we applied, as detection of codes depends on careful reading. The way we tried to ensure that as many instances were captured in the analysis

was via double coding and discussing all discrepancies. Additionally, our experience with e-voting is limited to the papers we have analyzed. Researchers with more experience in this field might have developed the scheme differently, as well as the search string based on what they know relevant papers would usually use for keywords.

Additionally, the inclusion or exclusion of papers may be error-prone due to subjectivity, as we have to assess whether it relates to e-voting and security enough to be included. However, both authors reviewed each paper and discussed any papers we were unsure about.

In relation to including security requirements that were not clearly implemented while coding, this choice might inflate certain requirements that are often discussed, but hard to implement. E.g. an article might say it is important to ensure coercion resistance, while also explaining that it is outside the scope of the paper to do so. This threatens *conclusion validity*. Although, those cases still reflect an interest or awareness that the security requirement is important. So, while our data cannot definitely say it only counts implemented security requirements, it still tracks interest in them.

In relation to our search string design, it is possible that we missed keywords that would have been relevant to our study, threatening *construct validity*. Our initial search was meant to limit this by first finding relevant papers and then adopting their keywords assuming other relevant papers would use the same keywords. As mentioned, during our snowballing we did find two relevant articles that were available on ACM Digital Library and Semantic Scholar respectively. This supports that the search string might have been too restrictive, and that more keywords were needed. We did use the e-vote-id proceedings as well as snowballing to find more relevant papers not caught by our systematic search to limit this issue, however, our snowballing was limited to only one paper.

8 Conclusion

We have conducted a systematic mapping study of e-voting literature, focusing on studies related to the security of e-voting between the period 2015 to 2025.
We analyzed the literature across 4 dimensions; technological architectures, security requirements, vulnerabilities, and paper types.
We found that Helios and Blockchain systems were frequently mentioned in the literature, with

Helios dropping in interest after 2019 and Blockchain becoming increasingly popular after 2017. The e-voting literature mostly focused on online voting systems, while most elections either use pure ballot voting or hybrid voting. This gap is likely due to coercion and other attacks like DoS being an obstacle to online voting, making it a more pressing research issue compared to hybrid systems with less pressing technical threats.

We found that the most mentioned security requirements, like privacy and integrity, were stable throughout all years, indicating that requirements are well-established and there is relative consensus as to what requirements are important. Based on the identified security requirements in the literature, we contributed with functional and non-functional requirements specifications as a practical reference for developers implementing e-voting systems.

The most frequently mentioned vulnerabilities were coercion and DoS, coercion being a major obstacle to adoption of online voting systems and DoS threatening the adoption of both online and hybrid e-voting systems for national elections.

For paper types, we found that most papers qualified as proposals (29%) and implementations (26%), with 19% of the papers focusing on some form of evaluations, i.e. case studies, user studies, and security analyses. This indicates a gap where a great share of the literature has focused on design/implementation of systems, compared to evaluation of systems.

Bibliography

[1] D. A. Gritzalis, "Principles and requirements for a secure e-voting system," *Computers & Security*, vol. 21, no. 6, pp. 539–556, 2002. https://doi.org/10.1016/S0167-4048(02)01014-3

[2] A. Benabdallah, A. Audras, L. Coudert, N. El Madhoun, and M. Badra, "Analysis of blockchain solutions for e-voting: A systematic literature review," *IEEE Access*, vol. 10, pp. 70746–70759, 2022. https://doi.org/10.1109/ACCESS.2022.3187688

[3] International IDEA, "Use of e-voting around the world," Feb. 6, 2023. [Online]. Available: https://www.idea.int/news-media/multimedia-reports/use-e-voting-around-world

[4] K. Petersen, R. Feldt, S. Mujtaba, and M. Mattsson, "Systematic mapping studies in software engineering," in *Proc. 12th Int. Conf. Evaluation and Assessment in Software Engineering*, 2008, p. 17.

[5] S. Sepulveda, M. Bustamante, and A. Cravero, "Identification of non-functional requirements for electronic voting systems: A systematic mapping," *IEEE Latin America Transactions*, vol. 13, no. 5, pp. 1577–1583, 2015. https://doi.org/10.1109/TLA.2015.7112018

[6] T. Aidynov, N. Goranin, D. Satybaldina, and A. Nurusheva, "A systematic literature review of current trends in electronic voting system protection using modern cryptography," *Applied Sciences*, vol. 14, no. 7, p. 2742, 2024. https://doi.org/10.3390/app14072742

[7] R. Barelli, M. D'Onghia, and S. Longari, "Toward secure electronic voting: A survey on e-voting systems and attacks," *IEEE Access*, vol. 13, pp. 89600–89626, 2025. https://doi.org/10.1109/ACCESS.2025.3569334

[8] U. Jafar, M. J. Ab Aziz, Z. Shukur, and H. A. Hussain, "A systematic literature review and meta-analysis on scalable blockchain-based electronic voting systems," *Sensors*, vol. 22, no. 19, p. 7585, 2022. https://doi.org/10.3390/s22197585

[9] W. Quesenbery, S. Chapman, C. Patten, and R. Spreggiaro, "A review of the literature on voter verification and ballot review," National Institute of Standards and Technology, Tech. Rep. NIST GCR 24-052, 2024. https://doi.org/10.6028/NIST.GCR.24-052

[10] R. Taş and Ö. Ö. Tanriöver, "A systematic review of challenges and opportunities of blockchain for e-voting," *Symmetry*, vol. 12, no. 8, p. 1328, 2020. https://doi.org/10.3390/sym12081328

[11] M. Alharby, A. Aldweesh, and A. van Moorsel, "Blockchain-based smart contracts: A systematic mapping study of academic research (2018)," in *Proc. 2018 Int. Conf. Cloud Computing, Big Data and Blockchain (ICCBB)*, 2018, pp. 1–6. https://doi.org/10.1109/ICCBB.2018.8756390

[12] Council of Europe, "Recommendation CM/Rec(2017)5 of the Committee of Ministers to member states on standards for e-voting," 2017. [Online]. Available: https://search.coe.int/cm#{%22CoEReference%22:[%22CM/Rec(2017)5%22],%22sort%22:[%22CoEValidationDate%20Descending%22],%22CoEIdentifier%22:[%2209125948802 2b2d5%22]}

[13] M. Bernhard, J. Benaloh, J. A. Halderman, R. L. Rivest, P. Y. A. Ryan, P. B. Stark, V. Teague, P. L. Vora, and D. S. Wallach, "Public evidence from secret ballots," in *Electronic Voting: E-Vote-ID 2017*, R. Krimmer, M. Volkamer, N. Braun Binder, N. Kersting, O. Pereira, and C. Schürmann, Eds., Lecture Notes in Computer Science, vol. 10615. Springer, 2017, pp. 84–109. https://doi.org/10.1007/978-3-319-68687-5_6

[14] M. Bruter, S. Harrison, E. Vives, and G. Gentile, "Study on e-voting practices in the EU," 2023.

[15] S. Bistarelli, M. Mantilacci, P. Santancini, and F. Santini, "An end-to-end voting-system based on Bitcoin," in *Proc. Symp. Applied Computing (SAC '17)*, New York, NY, USA, 2017, pp. 1836–1841. https://doi.org/10.1145/3019612.3019841

[16] A. Rodríguez-Pérez, P. Valletbó-Montfort, and J. Cucurull, "Bringing transparency and trust to elections: Using blockchains for the transmission and tabulation of results," in *Proc. 12th Int. Conf. Theory and Practice of Electronic Governance (ICEGOV '19)*, New York, NY, USA, 2019, pp. 46–55. https://doi.org/10.1145/3326365.3326372

[17] ACM Digital Library, "Search results," 2026. [Online]. Available: https://dl.acm.org/action/doSearch?AllField=%28%22electronic+voting%22+OR+%22e-voting%22+OR+%22internet+voting%22%29+AND+%28%22coercion+resistance%22+OR+%22verifiability%22+OR+%22integrity%22+OR+%22vulnerability%22+OR+%22attack%22+OR+%22usability%22+OR+%22human+factors%22%29

[18] Semantic Scholar, "Search results for electronic voting and related terms," 2026. [Online]. Available: https://www.semanticscholar.org/search?q=%28%22electronic%20voting%22%20OR%20%22e-voting%22%20OR%20%22internet%20voting%22%29%20AND%20%28%22coercion%20resistance%22%20OR%20%22verifiability%22%20OR%20%22integrity%22%20OR%20%22vulnerability%22%20OR%20%22attack%22%20OR%20%22usability%22%20OR%20%22human%20factors%22%29&sort=relevance

[19] dblp, "International Joint Conference on Electronic Voting (E-VOTE-ID)," 2026. [Online]. Available: https://dblp.org/db/conf/evoteid/index.html

[20] M. Lichtman, "Making meaning from your data," in *Qualitative Research in Education: A User's Guide*, SAGE, 2013, pp. 241–268.

[21] R. B. Büchter, A. Weise, and D. Pieper, "Development, testing and use of data extraction forms in systematic reviews: A review of methodological guidance," *BMC Medical Research Methodology*, vol. 20, Art. no. 259, 2020. https://doi.org/10.1186/s12874-020-01143-3

[22] N. Buscemi, L. Hartling, B. Vandermeer, L. Tjosvold, and T. P. Klassen, "Single data extraction generated more errors than double data extraction in systematic reviews," *Journal of Clinical Epidemiology*, vol. 59, no. 7, pp. 697–703, 2006. https://doi.org/10.1016/j.jclinepi.2005.11.010

[23] J. Horkoff, F. B. Aydemir, E. Cardoso, T. Li, A. Maté, E. Paja, M. Salnitri, L. Piras, J. Mylopoulos, and P. Giorgini, "Goal-oriented requirements engineering: An extended systematic mapping study," *Requirements Engineering*, vol. 24, pp. 133–160, 2019. https://doi.org/10.1007/s00766-017-0280-z

[24] V. Cortier, "Formal verification of e-voting: Solutions and challenges," *ACM SIGLOG News*, vol. 2, no. 1, pp. 25–34, Jan. 2015. https://doi.org/10.1145/2728816.2728823

[25] M. Backes, C. Hammer, D. Pfaff, and M. Skoruppa, "Implementation-level analysis of the JavaScript Helios voting client," in *Proc. 31st Annual ACM Symp. Applied Computing (SAC '16)*, New York, NY, USA, 2016, pp. 2071–2078. https://doi.org/10.1145/2851613.2851800

[26] N. Chang-Fong and A. Essex, "The cloudier side of cryptographic end-to-end verifiable voting: A security analysis of Helios," in *Proc. 32nd Annual Conf. Computer Security*

Applications (ACSAC '16), New York, NY, USA, 2016, pp. 324–335. https://doi.org/10.1145/2991079.2991106

[27] K.-H. Wang, S. K. Mondal, K. Chan, and X. Xie, "A review of contemporary e-voting: Requirements, technology, systems and usability," *Data Science and Pattern Recognition*, vol. 1, no. 1, pp. 31–47, 2017.

[28] D. Bernhard, O. Kulyk, and M. Volkamer, "Security proofs for participation privacy, receipt-freeness and ballot privacy for the Helios voting scheme," in *Proc. 12th Int. Conf. Availability, Reliability and Security (ARES '17)*, New York, NY, USA, 2017, Art. no. 1. https://doi.org/10.1145/3098954.3098990

[29] V. Cortier and J. Lallemand, "Voting: You can't have privacy without individual verifiability," in *Proc. 2018 ACM SIGSAC Conf. Computer and Communications Security (CCS '18)*, New York, NY, USA, 2018, pp. 53–66. https://doi.org/10.1145/3243734.3243762

[30] K. Marky, O. Kulyk, K. Renaud, and M. Volkamer, "What did I really vote for? On the usability of verifiable e-voting schemes," in *Proc. 2018 CHI Conf. Human Factors in Computing Systems (CHI '18)*, New York, NY, USA, 2018, Art. no. 176. https://doi.org/10.1145/3173574.3173750

[31] M. Meyer and B. Smyth, "Exploiting re-voting in the Helios election system," *Information Processing Letters*, vol. 143, pp. 14–19, 2019. https://doi.org/10.1016/j.ipl.2018.11.001

[32] T. Haines, R. Goré, and M. Tiwari, "Verified verifiers for verifying elections," in *Proc. 2019 ACM SIGSAC Conf. Computer and Communications Security (CCS '19)*, New York, NY, USA, 2019, pp. 685–702. https://doi.org/10.1145/3319535.3354247

[33] S. Baloglu, S. Bursuc, S. Mauw, and J. Pang, "Election verifiability revisited: Automated security proofs and attacks on Helios and Belenios," in *Proc. 2021 IEEE 34th Computer Security Foundations Symp. (CSF)*, Dubrovnik, Croatia, 2021, pp. 1–15. https://doi.org/10.1109/CSF51468.2021.00019

[34] K. Marky, M.-L. Zollinger, P. Roenne, P. Y. A. Ryan, T. Grube, and K. Kunze, "Investigating usability and user experience of individually verifiable internet voting schemes," *ACM Transactions on Computer-Human Interaction*, vol. 28, no. 5, Art. no. 30, 2021. https://doi.org/10.1145/3459604

[35] A. Kiayias, T. Zacharias, and B. Zhang, "DEMOS-2: Scalable E2E verifiable elections without random oracles," in *Proc. 22nd ACM SIGSAC Conf. Computer and Communications Security (CCS '15)*, New York, NY, USA, 2015, pp. 352–363. https://doi.org/10.1145/2810103.2813727

[36] P. Chaidos, V. Cortier, G. Fuchsbauer, and D. Galindo, "BeleniosRF: A non-interactive receipt-free electronic voting scheme," in *Proc. 2016 ACM SIGSAC Conf. Computer and Communications Security (CCS '16)*, New York, NY, USA, 2016, pp. 1614–1625. https://doi.org/10.1145/2976749.2978337

[37] T. Haines and X. Boyen, "VOTOR: Conceptually simple remote voting against tiny tyrants," in *Proc. Australasian Computer Science Week Multiconference (ACSW '16)*, New York, NY, USA, 2016, Art. no. 32. https://doi.org/10.1145/2843043.2843362

[38] J. Puiggalí, J. Cucurull, S. Guasch, and R. Krimmer, "Verifiability experiences in government online voting systems," in *Electronic Voting: E-Vote-ID 2017*, R. Krimmer, M. Volkamer, N. Braun Binder, N. Kersting, O. Pereira, and C. Schürmann, Eds., Lecture Notes in Computer Science, vol. 10615. Springer, 2017, pp. 248–263. https://doi.org/10.1007/978-3-319-68687-5

[39] H. Ge, S. Y. Chau, V. E. Gonsalves, H. Li, T. Wang, X. Zou, and N. Li, "Koinonia: Verifiable e-voting with long-term privacy," in *Proc. 35th Annual Computer Security Applications Conf. (ACSAC '19)*, New York, NY, USA, 2019, pp. 270–285. https://doi.org/10.1145/3359789.3359804

[40] M. Pawlak and A. Poniszewska-Marańda, "Blockchain e-voting system with the use of intelligent agent approach," in *Proc. 17th Int. Conf. Advances in Mobile Computing & Multimedia (MoMM2019)*, New York, NY, USA, 2020, pp. 145–154. https://doi.org/10.1145/3365921.3365927

[41] K. Marky, V. Zimmermann, M. Funk, J. Daubert, K. Bleck, and M. Mühlhäuser, "Improving the usability and UX of the Swiss internet voting interface," in *Proc. 2020 CHI Conf. Human Factors in Computing Systems*, 2020, pp. 1–13. https://doi.org/10.1145/3313831.3376769

[42] S. Baloglu, S. Bursuc, S. Mauw, and J. Pang, "Provably improving election verifiability in Belenios," in *Electronic Voting: E-Vote-ID 2021*, R. Krimmer, M. Volkamer, D. Duenas-Cid, O. Kulyk, P. Rønne, M. Solvak, and M. Germann, Eds., Lecture Notes in Computer Science, vol. 12900. Springer, 2021, pp. 1–16. https://doi.org/10.1007/978-3-030-86942-7

[43] M. Specter and J. A. Halderman, "Security analysis of the Democracy Live online voting system," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 3077–3092. https://www.usenix.org/conference/usenixsecurity21/presentation/specter-security

[44] S. Agbesi, J. Budurushi, A. Dalela, C. Nissen, and O. Kulyk, "How to increase transparency and trust in internet voting systems: An experimental study," in *Proc. 13th Nordic Conf. Human-Computer Interaction (NordiCHI '24)*, New York, NY, USA, 2024, Art. no. 29. https://doi.org/10.1145/3679318.3685362

[45] D. Chaum, R. T. Carback, J. Clark, C. Liu, M. Nejadgholi, B. Preneel, A. T. Sherman, M. Yaksetig, Z. Yin, F. Zagórski, and B. Zhang, "Revisiting silent coercion," in *Electronic Voting: E-Vote-ID 2025*, D. Duenas-Cid et al., Eds., Lecture Notes in Computer Science, vol. 16028. Springer, 2025, pp. 38–54. https://doi.org/10.1007/978-3-032-05036-6_3

[46] S. Shaheen, M. Yousaf, and M. Jalil, "Temper proof data distribution for universal verifiability and accuracy in electoral process using blockchain," in *Proc. 2017 13th Int. Conf. Emerging Technologies (ICET)*, 2017, pp. 1–6. https://doi.org/10.1109/ICET.2017.8281747

[47] M. Hajian Berenjestanaki, H. R. Barzegar, N. El Ioini, and C. Pahl, "Blockchain-based e-voting systems: A technology review," *Electronics*, vol. 13, no. 1, p. 17, 2024. https://doi.org/10.3390/electronics13010017

[48] S. Bartolucci, P. Bernat, and D. Joseph, "SHARVOT: Secret share-based voting on the blockchain," in *Proc. 1st Int. Workshop Emerging Trends in Software Engineering for Blockchain (WETSEB '18)*, New York, NY, USA, 2018, pp. 30–34. https://doi.org/10.1145/3194113.3194118

[49] I. L. Awalu, P. H. Kook, and J. S. Lim, "Development of a distributed blockchain eVoting system," in *Proc. 2019 10th Int. Conf. E-business, Management and Economics (ICEME '19)*, New York, NY, USA, 2019, pp. 207–216. https://doi.org/10.1145/3345035.3345080

[50] C. Killer, B. Rodrigues, R. Matile, E. Scheid, and B. Stiller, "Design and implementation of cast-as-intended verifiability for a blockchain-based voting system," in *Proc. 35th Annual ACM Symp. Applied Computing (SAC '20)*, New York, NY, USA, 2020, pp. 286–293. https://doi.org/10.1145/3341105.3373884

[51] P.-L. Wang, S.-H. Yang, and H.-C. Hsiao, "Hybrid-voting: A hybrid structured electronic voting system," in *Companion Proc. Web Conf. 2020 (WWW '20)*, New York, NY, USA, 2020, pp. 83–84. https://doi.org/10.1145/3366424.3382708

[52] A. Fatrah, S. El Kafhali, A. Haqiq, and K. Salah, "Proof of concept blockchain-based voting system," in *Proc. 4th Int. Conf. Big Data and Internet of Things (BDIoT '19)*, New York, NY, USA, 2020, Art. no. 31. https://doi.org/10.1145/3372938.3372969

[53] S. Park, M. Specter, N. Narula, and R. L. Rivest, "Going from bad to worse: From internet voting to blockchain voting," *Journal of Cybersecurity*, vol. 7, no. 1, p. tyaa025, 2021. https://doi.org/10.1093/cybsec/tyaa025

[54] P. McCorry, M. Mehrnezhad, E. Toreini, S. F. Shahandashti, and F. Hao, "On secure e-voting over blockchain," *Digital Threats*, vol. 2, no. 4, Art. no. 33, Dec. 2021. https://doi.org/10.1145/3461461

[55] M. P. Heinl, S. Gölz, and C. Bösch, "Remote electronic voting in uncontrolled environments: A classifying survey," *ACM Computing Surveys*, vol. 55, no. 8, Art. no. 167, Aug. 2023. https://doi.org/10.1145/3551386

[56] F. Rabia, A. Sara, and G. Taoufiq, "Review on blockchain-based e-voting systems," in *Proc. 2023 9th Int. Conf. Computer Technology Applications (ICCTA '23)*, New York, NY, USA, 2023, pp. 151–156. https://doi.org/10.1145/3605423.3605435

[57] S. Yousfi and M. Chaieb, "Building a scalable and coercion-resistant e-voting system using blockchain," *SN Computer Science*, vol. 6, p. 561, 2025. https://doi.org/10.1007/s42979-025-04088-w

[58] Y. Abuidris, R. Kumar, and W. Wenyong, "A survey of blockchain based on e-voting systems," in *Proc. 2019 2nd Int. Conf. Blockchain Technology and Applications (ICBTA '19)*, New York, NY, USA, 2020, pp. 99–104. https://doi.org/10.1145/3376044.3376060

[59] J. Cucurull, A. Rodríguez-Pérez, T. Finogina, and J. Puiggalí, "Blockchain-based internet voting: Systems' compliance with international standards," in *Business Information Systems Workshops. BIS 2018*, W. Abramowicz and A. Paschke, Eds., Lecture Notes in Business Information Processing, vol. 339. Springer, 2019. https://doi.org/10.1007/978-3-030-04849-5_27

[60] F. P. Hjálmarsson, G. K. Hreioarsson, M. Hamdaqa, and G. Hjálmtýsson, "Blockchain-based e-voting system," in *Proc. 2018 IEEE 11th Int. Conf. Cloud Computing (CLOUD)*, 2018, pp. 983–986. https://doi.org/10.1109/CLOUD.2018.00151

[61] R. Giustolisi and M. Sheikhi Garjan, "Efficient cleansing in coercion-resistant voting," in *Electronic Voting: E-Vote-ID 2024*, D. Duenas-Cid, P. Roenne, M. Volkamer, J. Budurushi,

M. Blom, A. Rodríguez-Pérez, I. Spycher-Krivonosova, J. Castellà Roca, and J. Barrat Esteve, Eds., Lecture Notes in Computer Science, vol. 15014. Springer, 2024, pp. 72–88. https://doi.org/10.1007/978-3-031-72244-8

[62] T. Haines, D. Pattinson, and M. Tiwari, "Verifiable homomorphic tallying for the Schulze vote counting scheme," in *Verified Software: Theories, Tools, and Experiments. VSTTE 2019*, S. Chakraborty and J. Navas, Eds., Lecture Notes in Computer Science, vol. 12031. Springer, 2020, pp. 36–53. https://doi.org/10.1007/978-3-030-41600-3_4

[63] C. Nissen, T. Hilt, J. Budurushi, M. Volkamer, and O. Kulyk, "Voting under pressure: Perceptions of counter-strategies in internet voting," in *Electronic Voting: E-Vote-ID 2025*, D. Duenas-Cid, P. Roenne, M. Volkamer, M. Blom, P. Gaudry, I. Borucki, and A. Debant, Eds., Lecture Notes in Computer Science, vol. 16028. Springer, 2025, pp. 158–174. https://doi.org/10.1007/978-3-032-05036-6

[64] T. Haines, J. Müller, and I. Querejeta-Azurmendi, "Scalable coercion-resistant e-voting under weaker trust assumptions," in *Proc. 38th ACM/SIGAPP Symp. Applied Computing (SAC '23)*, New York, NY, USA, 2023, pp. 1576–1584. https://doi.org/10.1145/3555776.3578730

[65] ACE Electoral Knowledge Network, "Direct recording electronically (DRE)," n.d. [Online]. Available: https://aceproject.org/ace-en/topics/et/eth/eth02/eth02b/eth02b3/mobile_browsing/onePag

[66] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. CRC Press, Taylor & Francis Group, 2021.

[67] D. F. Aranha, P. Y. S. Barbosa, T. N. C. Cardoso, C. L. Araújo, and P. Matias, "The return of software vulnerabilities in the Brazilian voting machine," *Computers & Security*, vol. 86, pp. 335–349, 2019. https://doi.org/10.1016/j.cose.2019.06.009

[68] T. Treier and K. Düüna, "Identifying and solving a vulnerability in the Estonian internet voting process: Subverting ballot integrity without detection," *IEEE Access*, vol. 12, pp. 197766–197782, 2024. https://doi.org/10.1109/ACCESS.2024.3521337

[69] C. Gallegos and D. Shin, "A novel device for secure home e-voting," in *Proc. 2015 Conf. Research in Adaptive and Convergent Systems (RACS '15)*, New York, NY, USA, 2015, pp. 321–323. https://doi.org/10.1145/2811411.2811541

[70] W. Jamroga and M. Tabatabaei, "Preventing coercion in e-voting: Be open and commit," in *Electronic Voting: E-Vote-ID 2016*, R. Krimmer, M. Volkamer, J. Barrat, J. Benaloh, N. Goodman, P. Y. A. Ryan, and V. Teague, Eds., Lecture Notes in Computer Science, vol. 10141. Springer, 2017, pp. 1–17. https://doi.org/10.1007/978-3-319-52240-1

[71] V. Cortier, N. Grimm, J. Lallemand, and M. Maffei, "A type system for privacy properties," in *Proc. 2017 ACM SIGSAC Conf. Computer and Communications Security (CCS '17)*, New York, NY, USA, 2017, pp. 409–423. https://doi.org/10.1145/3133956.3133998

[72] F. Belardinelli, R. Condurache, C. Dima, W. Jamroga, and A. V. Jones, "Bisimulations for verifying strategic abilities with an application to ThreeBallot," in *Proc. 16th Conf. Autonomous Agents and MultiAgent Systems (AAMAS '17)*, Richland, SC, USA, 2017, pp. 1286–1295.

[73] B. Zhang and H.-S. Zhou, "Brief announcement: Statement voting and liquid democracy," in *Proc. ACM Symp. Principles of Distributed Computing (PODC '17)*, New York, NY, USA, 2017, pp. 359–361. https://doi.org/10.1145/3087801.3087868

[74] L. J. Osterweil, M. Bishop, H. M. Conboy, H. Phan, B. I. Simidchieva, G. S. Avrunin, L. A. Clarke, and S. Peisert, "Iterative analysis to improve key properties of critical human-intensive processes: An election security example," *ACM Transactions on Privacy and Security*, vol. 20, no. 2, Art. no. 5, 2017. https://doi.org/10.1145/3041041

[75] R. del Pino, V. Lyubashevsky, G. Neven, and G. Seiler, "Practical quantum-safe voting from lattices," in *Proc. 2017 ACM SIGSAC Conf. Computer and Communications Security (CCS '17)*, New York, NY, USA, 2017, pp. 1565–1581. https://doi.org/10.1145/3133956.3134101

[76] S. Mödersheim and L. Viganò, "Alpha-beta privacy," *ACM Transactions on Privacy and Security*, vol. 22, no. 1, Art. no. 7, Feb. 2019. https://doi.org/10.1145/3289255

[77] S. Bag, M. A. Azad, and F. Hao, "E2E verifiable Borda count voting system without tallying authorities," in *Proc. 14th Int. Conf. Availability, Reliability and Security (ARES '19)*, New York, NY, USA, 2019, Art. no. 11. https://doi.org/10.1145/3339252.3339259

[78] S. M. T. Toapanta, L. Briones Peñafiel, and L. E. Mafla Gallegos, "Prototype to mitigate the risks of the integrity of cyberattack information in electoral processes in Latin America," in *Proc. 2019 2nd Int. Conf. Education Technology Management (ICETM '19)*, New York, NY, USA, 2020, pp. 111–118. https://doi.org/10.1145/3375900.3375915

[79] P. Grontas and A. Pagourtzis, "Anonymity and everlasting privacy in electronic voting," *International Journal of Information Security*, vol. 22, pp. 819–832, 2023. https://doi.org/10.1007/s10207-023-00666-2

[80] V. Zuevsky, "Proof of work and secure element in electronic voting," *ACRT*, vol. 2, Sep. 2023. https://doi.org/10.5772/acrt.28

[81] A. B. Pedin and N. Siasi, "Secure and decentralized anonymous e-voting scheme," in *Proc. 2023 ACM Southeast Conf. (ACMSE '23)*, New York, NY, USA, 2023, pp. 172–176. https://doi.org/10.1145/3564746.3587107

[82] S. Prajapat, U. Gautam, D. Gautam, P. Kumar, and A. V. Vasilakos, "Designing a robust quantum signature protocol based on quantum key distribution for e-voting applications," *Mathematics*, vol. 12, no. 16, p. 2558, 2024. https://doi.org/10.3390/math12162558

[83] I. Malik, B. Hu, and S. M. Noman, "E-voting as an e-government application: A foundation analysis," in *Proc. Int. Conf. Decision Science & Management (ICDSM '24)*, New York, NY, USA, 2024, pp. 258–263. https://doi.org/10.1145/3686081.3686125

[84] C. Culnane, P. Y. A. Ryan, S. Schneider, and V. Teague, "vVote: A verifiable voting system," *ACM Transactions on Information and Systems Security*, vol. 18, no. 1, Art. no. 3, 2015. https://doi.org/10.1145/2746338

[85] T. Miura, A. Kitagami, Y. Fujinawa, and T. Nagoya, "Accessibility, efficacy, and improvements in voting methodology for visually impaired persons using a web-based electronic ballot system," in *Proc. 8th Indian Conf. Human-Computer Interaction (IndiaHCI '16)*, New York, NY, USA, 2016, pp. 75–83. https://doi.org/10.1145/3014362.3014370

[86] L. Babenko, I. Pisarev, and O. Makarevich, "A model of a secure electronic voting system based on blind intermediaries using Russian cryptographic algorithms," in *Proc. 10th Int. Conf. Security of Information and Networks (SIN '17)*, New York, NY, USA, 2017, pp. 45–50. https://doi.org/10.1145/3136825.3136876

[87] L. Babenko and I. Pisarev, "Distributed e-voting system based on blind intermediaries using homomorphic encryption," in *Proc. 11th Int. Conf. Security of Information and Networks (SIN '18)*, New York, NY, USA, 2018, Art. no. 6. https://doi.org/10.1145/3264437.3264473

[88] L. Zahhafi and O. Khadir, "Anonymous secure e-voting over a network for multiple elections," in *Proc. 2nd Int. Conf. Networking, Information Systems & Security (NISS '19)*, New York, NY, USA, 2019, Art. no. 46. https://doi.org/10.1145/3320326.3320378

[89] L. Babenko, I. Pisarev, and E. Popova, "Cryptographic protocols implementation security verification of the electronic voting system based on blind intermediaries," in *Proc. 12th Int. Conf. Security of Information and Networks (SIN '19)*, New York, NY, USA, 2019, Art. no. 27. https://doi.org/10.1145/3357613.3357641

[90] N. Akinyokun and V. Teague, "Receipt-free, universally and individually verifiable poll attendance," in *Proc. Australasian Computer Science Week Multiconference (ACSW '19)*, New York, NY, USA, 2019, Art. no. 5. https://doi.org/10.1145/3290688.3290696

[91] M. Tabatabaei, W. Jamroga, and P. Y. A. Ryan, "Expressing receipt-freeness and coercion-resistance in logics of strategic ability: Preliminary attempt," in *Proc. 1st Int. Workshop AI for Privacy and Security (PrAISe '16)*, New York, NY, USA, 2016, Art. no. 1. https://doi.org/10.1145/2970030.2970039

[92] S. Hagihara, M. Shimakawa, and N. Yonezaki, "Verification of verifiability of voting protocols by strand space analysis," in *Proc. 2019 8th Int. Conf. Software and Computer Applications (ICSCA '19)*, New York, NY, USA, 2019, pp. 363–368. https://doi.org/10.1145/3316615.3316629

[93] E. Estaji, T. Haines, K. Gjøsteen, P. B. Rønne, P. Y. A. Ryan, and N. Soroush, "Revisiting practical and usable coercion-resistant remote e-voting," in *Electronic Voting: E-Vote-ID 2020*, R. Krimmer, M. Volkamer, B. Beckert, R. Küsters, O. Kulyk, D. Duenas-Cid, and M. Solvak, Eds., Lecture Notes in Computer Science, vol. 12455. Springer, 2020, pp. 50–66. https://doi.org/10.1007/978-3-030-60347-2

[94] S. M. T. Toapanta, M. I. Sinche, and L. E. Mafla Gallegos, "A cyber environment approach to mitigate vulnerabilities and threats in an electoral process in Ecuador," in *Proc. 2019 2nd Int. Conf. Education Technology Management (ICETM '19)*, New York, NY, USA, 2020, pp. 97–104. https://doi.org/10.1145/3375900.3375914

[95] Y. Erb, D. Duenas-Cid, and M. Volkamer, "Identifying factors studied for voter trust in e-voting – Review of literature," in *Electronic Voting: E-Vote-ID 2023*, M. Volkamer, D. Duenas-Cid, P. Rønne, P. Y. A. Ryan, J. Budurushi, O. Kulyk, A. Rodriguez Pérez, and I. Spycher-Krivonosova, Eds., Lecture Notes in Computer Science, vol. 14230. Springer, 2023, pp. 93–117. https://doi.org/10.18420/e-vote-id2023_16

[96] S. Agbesi, J. Budurushi, A. Dalela, and O. Kulyk, "Investigating transparency dimensions for internet voting," in *Electronic Voting: E-Vote-ID 2023*, M. Volkamer, D. Duenas-Cid, P. Rønne, P. Y. A. Ryan, J. Budurushi, O. Kulyk, A. Rodriguez Pérez, and I. Spycher-Krivonosova, Eds., Lecture Notes in Computer Science, vol. 14230. Springer, 2023, pp. 1–17. https://doi.org/10.1007/978-3-031-43756-4

[97] A. Conway, M. Blom, L. Naish, and V. Teague, "An analysis of New South Wales electronic vote counting," in *Proc. Australasian Computer Science Week Multiconference (ACSW '17)*, New York, NY, USA, 2017, Art. no. 24. https://doi.org/10.1145/3014812.3014837

[98] D. Basin, H. Gersbach, A. Mamageishvili, L. Schmid, and O. Tejada, "Election security and economics: It's all about Eve," in *Electronic Voting: E-Vote-ID 2017*, R. Krimmer, M. Volkamer, N. Braun Binder, N. Kersting, O. Pereira, and C. Schürmann, Eds., Lecture Notes in Computer Science, vol. 10615. Springer, 2017, pp. 1–20. https://doi.org/10.1007/978-3-319-68687-5

[99] L. Babenko and I. Pisarev, "Modeling replay and integrity violations attacks for cryptographic protocols source codes verification of e-voting system based on blind intermediaries," in *Proc. 13th Int. Conf. Security of Information and Networks (SIN 2020)*, New York, NY, USA, 2021, Art. no. 26. https://doi.org/10.1145/3433174.3433597

[100] J. Vakarjuk, N. Snetkov, and J. Willemson, "Russian federal remote e-voting scheme of 2021 – Protocol description and analysis," in *Proc. 2022 European Interdisciplinary Cybersecurity Conf. (EICC '22)*, New York, NY, USA, 2022, pp. 29–35. https://doi.org/10.1145/3528580.3528586

[101] M. Bernhard, A. McDonald, H. Meng, J. Hwa, N. Bajaj, K. Chang, and J. A. Halderman, "Can voters detect malicious manipulation of ballot marking devices?" in *Proc. 2020 IEEE Symp. Security and Privacy (SP)*, San Francisco, CA, USA, 2020, pp. 679–694. https://doi.org/10.1109/SP40000.2020.00118

[102] L. Cristiano, R. Longo, and C. Spadafora, "Click and cast: Assessing the usability of Vote App," in *Electronic Voting: E-Vote-ID 2024*, D. Duenas-Cid, P. Roenne, M. Volkamer, J. Budurushi, M. Blom, A. Rodríguez-Pérez, I. Spycher-Krivonosova, J. Castellà Roca, and J. Barrat Esteve, Eds., Lecture Notes in Computer Science, vol. 15014. Springer, 2024, pp. 67–100. https://doi.org/10.18420/e-vote-id2024_05

[103] F. Verity and D. Pattinson, "Formally verified invariants of vote counting schemes," in *Proc. Australasian Computer Science Week Multiconference (ACSW '17)*, New York, NY, USA, 2017, Art. no. 31. https://doi.org/10.1145/3014812.3014845

[104] D. Duenas-Cid, "Trust and distrust in electoral technologies: What can we learn from the failure of electronic voting in the Netherlands (2006/07)," in *Proc. 25th Annual Int. Conf. Digital Government Research (dg.o '24)*, New York, NY, USA, 2024, pp. 669–677. https://doi.org/10.1145/3657054.3657262

[105] P. Parycek, M. Sachs, S. Virkar, and R. Krimmer, "Voting in e-participation: A set of requirements to support accountability and trust by electoral committees," in *Electronic Voting: E-Vote-ID 2017*, R. Krimmer, M. Volkamer, N. Braun Binder, N. Kersting, O. Pereira, and C. Schürmann, Eds., Lecture Notes in Computer Science, vol. 10615. Springer, 2017, pp. 42–56. https://doi.org/10.1007/978-3-319-68687-5_6

[106] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2024. https://doi.org/10.1007/978-3-662-69306-3

[107] National Academies of Sciences, Engineering, and Medicine, *Securing the Vote: Protecting American Democracy*. National Academies Press, 2018.

[108] ACE Electoral Knowledge Network, "How do electors cast their votes?" 2026. [Online]. Available: https://aceproject.org/epic-en?question=VO011

Appendices:

Appendix 1

Paper type coding scheme

Type	Definition
Proposal	A publication that proposes something novel, including new voting technology, schemes, algorithms etc. New evaluations of e.g. a voting implementation (like a case study) was not counted as proposals. We did not judge whether the article proposed something genuinely new, only that the paper described it as such. Code definition based on previous work from Horkoff et al. [23, p. 138].
Formalization	If the article contained a formal proof, we counted it as having a formalization. We also included formal proofs about insecurity of a system under this definition. However, it does not include attacks on implemented systems as part of a security analysis, in those cases use the "security analysis" code. Code definition inspired by Horkoff et al. [23, p. 138], but with a specific focus on formalizations related to e-voting.
Meta-study	If the article provided an extensive overview of previous work beyond what you would find in a typical related work section, we coded it as a meta-study. Code definition based on previous work from Horkoff et al. [23, p. 138].
Implementation	Publications that mention building a tool/e-voting system either in the form of pseudocode or actual code. It was not required that the tool was implemented by the paper authors. Code definition based on previous work from Horkoff et al. [23, p. 138].
Case study	The publication includes a case study either as part of the contribution of the paper or as data

	analyzed in the paper. Code definition based on previous work from Horkoff et al. [23, p. 138].
Security analysis	The publication studies an e-voting system's security by e.g. trying to exploit it. We did not count pure formal verifications of systems in this category, instead focusing on cases where an implemented system was analyzed.
User study	The publication conducts a study with participants, either conducting tests of e.g. an e-voting system or using other qualitative methods like semi-structured interviews to gain insights into use of e-voting systems.
Regulatory analysis	The publication analyzes and/or synthesizes regulatory texts.

Appendix 2

Security requirement coding scheme

Security requirement	**Definition**	**Examples**
Software independence	This code is used for cases where software independence or strong software independence is discussed or implemented as a security requirement [13, p.88]. By software independence, we refer to the requirement that if an undetected change or error occurs in a voting system's software, it should not cause a change or error in the election outcome (Ibid.).	"This system satisfies strong software independence…" "In order to achieve software independence, some states utilize permanent paper records"
End-to-end verifiability	This code is used for cases where end-to-end verifiability is discussed or implemented as a security requirement [13, p. 89]. Using Bernhard et al. definition, we define end-to-	"This system satisfies both individual and universal verifiability, thus ensuring end-to-end verifiability"

	end verifiability as consisting of cast-as-intended, collected-as-cast, and counted-as-recorded verifiability (Ibid.). Note that this code should not be used if only one or two of cast-as-intended, collected-as-cast, and counted-as-recorded verifiability are mentioned. If all three apply codes would apply for a single system in the paper, code "end-to-end verifiability" instead. End-to-end verifiability should not be used for other types of verifiability. In those cases, use the appropriate "verifiability" code.	"This system guarantees e2e verifiability"
Collection accountability	This code is used for cases where collection accountability is discussed or implemented as a security requirement [13, p.89]. If a voter can provide convincing evidence that their vote has not been collected in legitimate cases, then the system has collection accountability (Ibid.).	"It should be possible for a voter to contact the election committee in case there has been an error with their vote."
Dispute resolution	This code is used for cases where dispute resolution is discussed or implemented as a security requirement [13, p.89]. Dispute resolution covers systems where a dispute between two participants can be resolved by a third party. This code also covers systems that are dispute free, meaning the system has built-in preventions that eliminate disputes between participants and that any third party can see	"Given that our system utilizes a ballot confirmation check that returns a valid proof if the vote is valid, it can help with dispute resolutions between multiple parties."

	if a participant has cheated (Ibid.).	
Verifiability • Verifiability (cast-as-intended) • Verifiability (recorded-as-cast) • Verifiability (counted-as-recorded) • Verifiability (other)	This code is used for cases where verifiability and more specific versions of verifiability are discussed or implemented as a security requirement [13, p.89]. When the case mentions a specific type of verifiability, then use the appropriate subcode. If a case is verifiability, but no appropriate subcode is mentioned, use verifiability (other). If the case mentions end-to-end verifiability, use the "end-to-end verifiability" code instead. verifiability (cast-as-intended) - use for cases where cast-as-intended verifiability is discussed or implemented as a security requirement. A system has cast-as-intended verifiability if voters can verify that their cast ballot is correctly recorded (Ibid.). verifiability (recorded-as-cast) - use for cases where recorded-as-cast verifiability is discussed or implemented as a security requirement. A system has recorded-as-cast verifiability if voters can verify that their vote is collected properly in the tally (Ibid.). verifiability (counted-as-recorded) - use for cases where counted-as-recorded verifiability is discussed or implemented as a security requirement. A system has	"Our system ensures individual verifiability" - code both "verifiability (cast-as-intended)" and "verifiability (recorded-as-cast)" since individual verifiability is usually defined as a combination of cast-as-intended and recorded-as-cast verifiability, i.e. end-to-end verifiability without counted-as-recorded verifiability (King-Hang et al., 2017, p. 34). "Our system guarantees cast-as-intended verifiability." - code "verifiability (cast-as-intended)". "Our system guarantees recorded-as-cast verifiability." - code "verifiability (recorded-as-cast)". "Our system ensures universal verifiability" - code "verifiability (counted-as-recorded)" since universal verifiability is also used to describe counted-as-recorded verifiability (King-Hang et al., 2017, p. 34). "Our system ensures counted-as-recorded verifiability" - code "verifiability (counted-as-recorded)". "Using this technique we ensure that our system has verifiability" - code "verifiability

	counted-as-recorded verifiability if anyone can verify that every correctly cast and collected vote is included in the tally (Ibid.). verifiability (other) - use for cases where verifiability is discussed or implemented as a security requirement, but it is not captured by any of the other subcodes. For example, when it is not clear what type of verifiability is discussed.	(other)" since the exact type of verifiability is not specified.
Everlasting privacy	This code is used for cases where everlasting privacy or long-lasting privacy is discussed or implemented as a security requirement [13, p. 93]. Cases that describe systems that do not rely on cryptographic hardness to protect privacy fall under this category (Ibid.). Do not use this code when the security requirement is about weaker notions of privacy, in that case use the code "privacy".	"Everlasting privacy is ensured in our system using..." "This scheme ensures long-lasting privacy in relation to quantum adversaries"
Receipt-freeness	This code is used for cases where receipt-freeness is discussed or implemented as a security requirement [13, p.94]. A system is receipt free if a voter cannot prove how they wanted, even if they wanted to (Ibid.).	"Our system ensures receipt-freeness".
Coercion resistance	This code is used for cases where coercion resistance or uncoercionability is discussed or implemented as a security requirement [13, pp.94-95]. We define a system as coercion resistant if, given any strategy a coercer may employ, there is a	"Using this technique we can ensure coercion-resistance"

	way for a voter to vote in a way that is not identifiable to a coercer, enabling them to still cast their vote (Ibid.).	
Availability	This code is used for cases where availability is discussed or implemented as a security requirement [13, pp.95-96]. A hybrid or fully electronic voting system satisfies availability if it can resist attempts to disrupt the election process; both partial disruptions like selective denial of service (DoS) attacks as well as complete disruptions (Ibid.).	"In order to ensure availability, we have developed a decentralized system..."
Usability • Usability (efficiency) • Usability (effectiveness) • Usability (satisfaction) • Usability (transparency) • Usability (other)	This category of codes are used for cases where usability is discussed or implemented as a security requirement [13, pp.96-97]. Usability can be discussed as its own security requirement or in a more specific form like efficiency, effectiveness, satisfaction, and transparency. Below are subcodes to classify what type of usability is discussed. usability (efficiency) - use for cases where efficiency is argued as important to security and a requirement of the system. If the case does not relate efficiency to security, then do not code it. usability (effectiveness) - use for cases where effectiveness is argued as important to security and a requirement of the system. If the case does not relate effectiveness to security, then do not code it.	"Transparency is important to ensure correct use of the system" - code "usability (transparency)". "Usability is a core requirement in our system, the system needs to be usable to ensure..."

	usability (satisfaction) - use for cases where satisfaction is argued as important to security and a requirement of the system. If the case does not relate satisfaction to security, then do not code it. usability (transparency) - use for cases where information about the system needs to be transparent and accessible to users. usability (other) - use for cases where usability is mentioned or when there is a security requirement that relates directly to usability, but there is no appropriate subcode to describe it. If the case does not relate usability to security, then do not code it.	
Auditability	This code is used for cases where auditing is discussed or implemented as a security requirement [13, p.98].	"Auditing of the vote is a fixed step in our system, it is important to confirm the validity of the election results."
Correctness	This code is used for cases where correctness is discussed or implemented as a security requirement (Wang et al., 2017, p. 33). Correctness in this case refers to all votes being correctly counted, meaning all valid votes are counted, and all invalid votes are not counted (Ibid.). Note that general mentions of correctness, such as correctness of a proof should not be coded as correctness.	"... this property is needed to ensure correctness of the system".
Privacy	This code is used for cases where privacy is discussed or implemented as a security	"... to ensure ballot secrecy..."

	requirement (Wang et al., 2017, p. 33). A system ensures privacy if the voter's ballot choices are only known to themselves (Ibid.). Systems that ensure privacy via cryptographic hardness assumptions are included. If a system does not rely on such assumptions to ensure privacy code "everlasting privacy" instead.	"This would give immediate privacy..." "This ensures confidentiality of the data, only the voter can see their results" "This ensures that our system has privacy."
Unreuseability	This code is used for cases where unreuseability, the property that no voter can cast a vote twice, is discussed or implemented as a security requirement (Wang et al., 2017, p. 33).	"Our system ensures unreuseability." "No voter can cast their vote more than once."
Eligibility	This code is used for cases where eligibility, the property that only authorized voters can vote, is discussed or implemented as a security requirement (Wang et al., 2017, p. 33). We also include cases where eligibility is made verifiable for the individual or where independent parties can verify all voters are eligible.	"Our system ensures eligibility via..." "Only authorized voters are permitted to cast a vote."
Robustness	This code is used for cases where robustness is discussed or implemented as a security requirement (Wang et al., 2017, p. 33). A system has robustness if it can properly function even with some number of misbehaved voters or partial system failure (Ibid.). We do not assess whether a	"Our system ensures robustness." "The system remains functional even in the case of partial system failure or misbehaving voters."

	system satisfies a specific threshold for misbehaved voters or system failure, instead assess if the system is robust by the standards used in the specific case. For example, if the author(s) in a paper argues the system satisfies robustness, we apply the code.	
Fairness	This code is used for cases where fairness is discussed or implemented as a security requirement (Wang et al., 2017, p. 33). This code should only be used in cases where the property of only counting the vote once all votes have been corrected such that no election authority can compute partial election results before the end of the election (Ibid.). General mentions of fairness, if not related to this requirement should not be coded.	“The system ensures fairness properties…” "The election authority cannot compute partial results before the election has ended."
Practicality	This code is used for cases where the requirement that the system should not rely on difficult to realize assumptions in large-scale scenarios, is discussed or implemented (Wang et al., 2017, p. 34).	“It is important that the system can be used for smaller elections.” “… that scheme used difficult to realize assumptions whereas we have designed a protocol with weaker assumptions that can be more realistically used in live systems…”
Safe aggregation	This code is used for cases where there is a requirement of aggregating votes from electronic and non-electronic channels and to safely compute them in hybrid	"Our system ensures safe aggregation." "Votes cast through electronic and non-electronic channels

	elections, is discussed or implemented (Cucurull et al., 2019, p. 302).	are securely combined into a single tally."
Authentication	This code is used for cases where authentication is discussed or implemented as a security requirement (Cucurull et al., 2019, p. 302). A system provides authentication if it can uniquely identify voters, and if it only authorizes eligible voters to access the vote (Ibid.).	"To ensure authentication, our system uses..." "Only authorized voters are permitted to access the ballot..." “Voters are uniquely identified via their unique ID, ensuring authentication...”
Integrity	This code is used for cases where integrity is discussed or implemented as a security requirement (Cucurull et al., 2019, p. 303). Integrity here is understood as the voter, and more broadly the election result, not being affected by the voting system or any other actor with the intent to unduly sway the result (Ibid.).	"Our system ensures integrity by..."
Forward secrecy	This code used for cases where forward secrecy is discussed or implemented as a security requirement (Haines and Xavier, 2016, p.2).	"Our system ensures forward secrecy." "Past votes remain protected even if cryptographic keys are compromised in the future."
Non-valid voting capability	This code is used for cases where the requirement to allow voters to send invalid votes without compromising the integrity of the election is discussed or implemented as a security requirement (Hajian Berenjestanaki, 2024, p. 4).	"Voters can submit invalid votes without compromising the integrity of the election."
Anonymity	This code is used for cases where anonymity is discussed	"Our system ensures anonymity."

	or implemented as a security requirement (Cucurull et al., 2019, p. 303). A system ensures anonymity if it is not possible to link an unsealed vote to a voter (Ibid.).	"It is not possible to link a vote back to the voter who cast it."
Revocation	This code is used for cases where revocation is discussed or implemented as a security requirement (Haines & Boyen, 2016, pp. 6-7). A system allows for revocation if the voter can cancel or override their previously cast vote during the election period (Haines & Boyen, 2016, pp. 6–7).	"Our system supports revocation." "Voters can cancel or override their previously cast vote during the election period."
Legal requirement	This code is used for cases where legal requirements are considered, specifically those related to security. This includes cases where the publication analyzes applicable law, state it as a requirement for the voting system(s) in question, or when it is simply acknowledged that local law can and/or will affect security requirements in an e-voting system.	“Local law mandates that all cast votes must remain private..." “Law requires that all DREs have an attached printer so that a paper audit trail can be done at all times" “It is important to consider the legal context the system has to operate in as it affects how e.g. verification has to be carried out.”

Appendix 3

Vulnerability coding scheme

Vulnerability	Definition	Examples
Authentication attack	This code is used for general descriptions of attacks that either break authentication and/or exploit authentication	"We found that the authentication mechanism could be exploited to break privacy."

		features of a system to break other security requirements.	
Ballot reordering attack		This code is used for an adversary to manipulate the voting sequence to ensure a discarded or overwritten vote is counted instead of the voter's final, intended choice. This exploit targets systems that allow re-voting, where the rule is typically that only the last ballot cast is valid. (Baloglu et al., 2021 "PROVABLY IMPROVING...", p. 7	"Systems that allow re-voting are vulnerable to ballot reordering attacks."
Network attack		This code is used for exploiting unauthorized access, stealing data, or damage systems. The attack could originate from external or internal actors (Aljabri et al., 2021, p. 4).	"The system was found to be vulnerable to network attacks from both internal and external actors."
Hash-based attack		This code is used for attacks on hashes. A hash-based attack occurs when an adversary exploits a hash function that lacks collision resistance. [66, p. 168]	"Schemes that lack collision resistance are susceptible to hash-based attacks."
Quantum attacks		This code is used for all mention of attacks enabled by quantum computers, including e.g. brute forcing attacks specifically possible given quantum technology.	"... using a quantum computer even these cryptographic schemes could be broken via brute-forcing."
Brute-forcing attack		This code is used for an adversary gaining physical access to a voter's smart card and systematically attempting to guess the PIN to cast a	"The short PIN requirement makes the system vulnerable to brute-forcing attacks."

		fraudulent vote. Because PIN codes must be short enough for humans to remember, the search space is limited, making it possible for an attacker to automate the process and eventually find the correct combination. (Estaji et al., 2020, p. 54)	
Coercion attack		This code is used for general mentions of coercion. The code also includes some named attacks that were mentioned infrequently only once where the main aim of the attack is to enable coercion. Examples of attacks that fall under the category of coercion attack: vote buying [24], forced-abstention attacks, randomization attacks, and simulation attacks (Heinl et al., 2022, p.9).	"We found that the system was vulnerable to coercion attacks, including vote buying." “These types of schemes are still vulnerable to forced-abstention attacks.” “… but this property does not defend against randomization or simulation attacks. To protect the system against these types of attacks…”
Ballot stuffing attack		This code is used for an adversary to cast a false vote using a valid voter identity. This attack directly targets the property of Result Integrity. (Baloglu et al., 2021 “PROVABLY IMPROVING…”, p. 6)	“This means the system is vulnerable to ballot stuffing…”
Ballot replay attack		This code allows an attacker to capture an encrypted vote or a login token, and then send it again to the recipient to perform an unauthorized	"Systems that do not invalidate previously cast votes are susceptible to ballot replay attacks."

		action (Babenko & Pisarev, 2021, p. 2)	
Poisoning attack		This code enables an attack where a malicious voter submits a fake or corrupted ballot that is mathematically flawed. This type of attack doesn't try to change votes but instead breaks the system's ability to count any votes (Chang-Fong & Essex, 2016, p. 4)	"An unsatisfied voter could send in a corrupted ballot which could make decryption of the results impossible." "… this is vulnerable to attacks aiming at invalidating the vote using malicious ballots."
Denial-of-service attack		This code is used where an attacker floods a voting or registration server with fake traffic. This malicious overload causes the server to slow down or crash. (Toapanta et al., 2020, p. 114)	"DoS attacks are still a problem for most blockchain systems in the literature, threatening availability." "One attack which can disrupt the election is a Denial of Service attack"
Single point of failure		This code is used for attacks that rely on a design flaw where there is a trusted component in the system which trivializes an attack on the system. For example, Estaji et al., 2020, describes a protocol that utilizes smart cards for voting which has the security assumption that the smart card is trusted and not modified to be malicious (p. 57).	"The reliance on a trusted smart card introduces a single point of failure in the system." "To avoid a single point of failure the system utilizes…" "This presents a single point of failure where the adversary only needs to guess the pin, which is easy to do using bruteforcing"
Receipt attack		This code describes attacks where coerced voters are forced to vote in a way where the coercer can verify that they followed their instruction.	"This scheme is vulnerable to the Italian attack." "This is vulnerable to ballot copying…"

		This includes the italian attack, where the voter is instructed to vote with a pattern (e.g. some kind of personal identifier) that makes whether they followed the coercers instructions verifiable [13]. We also include some cases of ballot copying attacks [24], since the coercer can verify whether the voter has followed their instructions.	"This scheme is vulnerable to receipt attacks"
Defaming attack		This code is used for attacks where the adversary pretends that a system error has occurred by e.g. fabricating a vote receipt and then claiming it was cast but was omitted from the Web Bulleting Board (Culnane et al., 2015, p. 8).	"By fabricating a vote receipt, an adversary could perform a defaming attack against the election authority."
Kleptographic attack		This code is used to describe kleptographic attacks, where a backdoor is inserted into e.g. a cryptographic algorithm that allows an adversary to steal confidential information (Sajjad et al., 2020).	"Schemes that do not protect against backdoors are susceptible to kleptographic attacks."
Sybil attack		This code is used for attacks where the voter attempts to gain access to multiple identities to modify the election (Bartolucci et al, 2018, p. 31). Examples include making multiple pseudonyms (i.e. public keys) on the blockchain to vote (Ibid.).	“The blockchain system is vulnerable to a Sybil attack, as you can always generate multiple public keys to sign up with”

Malware		This code is used for attacks that use malware to infect a voter server. The attacker can then compromise the server and system and damage vital information.	"We found that the voting server was vulnerable to malware, compromising stored ballots."
Social engineering		This code is used for a variety of attacks with the common factor that they aim at exploiting human factors in the system (Hatfield, 2018). This code includes phishing (Khonji, 2013), impersonation of the voter and/or election authority, and some types of spoofing (Hatfield, 2018, p. 107).	“Fraud polling stations is a problem we have seen in…” “Voters could be exposed to phishing were the adversary impersonates an election authority…”
Injection attack		This code is used for vulnerabilities like integer overflows, SQL Injections which manipulate database queries to corrupt data, and XSS which inserts malicious scripts to gain authorized access or modify database content.	"We found that the system was vulnerable to SQL injection, allowing an attacker to manipulate the voting database." “In this study we show that the system is vulnerable to a Cross-Site-Scripting (XSS) attack…” “In our literature review we found the following vulnerabilities in e-voting systems: ballot copying attack, social engineering, inter overflow, man-in-the-midde attacks…”

Insider attack		This code is used for exploits that are characterized by being used by an insider in the system. An insider could be a corrupt election authority or a secret share holder in a system using, for example, Shamir's Secret Sharing [66, p.541].	"The system is susceptible to a last voter's share attack." "A corrupt election authority could potentially publish the results partway."
Tamper attack • tamper attack (other) • tamper attack (ballot)		This code is used for general mentions of tampering as well as cases where the ballot is being tampered with. If it is not clear what the object that is being tampered with is then code "tamper attack (other)". If tampering is specifically directed at modifying the ballot, then we use the subcode "tamper attack (ballot)". If hardware is tampered with, use the "hardware exploitation" code instead.	"… allowing us to tamper with the ballots" - code "tamper attack (ballot)" since it specifically mentions the ballot is tampered with. "The system was found to be vulnerable to tampering…"
Ballot design manipulation		This code is used for attacks where the attacker can change the ballot options, e.g. the candidates or the voter's options in terms of what they can select to manipulate the outcome (Specter and Halderman, 2021, p.8-10).	"This scheme is vulnerable to ballot design manipulation, allowing an attacker to modify the list of candidates."
Ballot misdirection attack		This code is used for attacks where the return instructions of the ballot are modified, causing the voted ballots to be	"We found that we could change the return instructions of the ballot, which could be exploited to delay the vote, potentially until after the election."

		sent to the wrong sorting location or delayed until after the vote (Specter and Halderman, 2021, p.8).	
Forgery attack		This code is used for mentions of forgery as an attack on a system, this includes credential forgery and vote forgery. When a more specific subcode better explains the exploit used such as “ballot replay attack”, then we code that instead.	"The scheme was found to be vulnerable to forgery attacks, allowing an adversary to forge voter credentials."
Hardware exploitation		This code is used for all mentions of exploiting hardware for elections. This includes mentions of tampering where the target is hardware, or an exploit that specifically targets a physical device.	“We found that we could tamper with the screen contents of the DRE.” “It was possible to hack into the DRE and upload malware”.
Clash attack		This code is used for all mentions of clash attacks, attacks where a voting machine attempts to provide different voters the same voting receipt, allowing election authorities to replace the identical receipts with new ballots affecting the election (Kusters et al., 2012).	"This scheme is vulnerable to clash attacks, as the voting machine can issue identical receipts to different voters."
Man-in-the-middle attack		This code is used for mentions of man-in-the-middle attacks, attacks where an adversary intercepts communications between two parties with the aim of stealing confidential	“In our literature review we found the following vulnerabilities in e-voting systems: ballot copying attack, social engineering, inter overflow, man-in-the-middle attacks…”

		information (Regenscheid, n.d.)	
Timing attack		This code is used for mentions of timing attacks, attacks that aim to gain confidential information about e.g. a decryption algorithm by timing how long it takes to perform certain steps (Crosby et al., 2009). This includes direct mentions of timing attacks as well as described attacks that utilize the premise of timing algorithms to gain information used for an exploit.	"This decryption scheme is susceptible to timing attacks, potentially leaking information about private keys."
Zero-day attack		This code is used for mentions of zero-day attacks, attacks that are unknown to the developers (Ahmad et al., 2023). We only coded "zero-day attack" if the paper used the literal term.	"One of the identified problems in the literature was zero-day attacks..."

Appendix 4

All functional and non-functional requirements:

Software independence:

- **FR1:** The system shall produce a voter-verifiable paper audit trail (VVAT) for every cast vote, such that election outcomes can be audited independently of the software.
- **NFR1:** The paper audit trail shall remain the authoritative record of the election outcome; any discrepancy between the software tally and the physical record shall trigger an automatic audit halt.

End-to-end verifiability:

- **FR1:** The system shall provide each voter with a unique tracking code after casting their vote, which they can use to verify that their vote appears correctly in the published tally.
- **FR2:** The system shall publish a cryptographically signed bulletin board containing all cast ballots and the final tally, accessible to any third-party auditor.

- **NFR1:** The system's verifiability mechanisms shall function correctly regardless of the underlying software or hardware used to conduct the election, such that no undetected software error can produce an undetectable change in the verified outcome.

Collection Accountability

- **FR1:** The system shall generate a timestamped confirmation receipt for each submitted vote, which the voter can use as evidence of submission if their vote is not reflected in the tally.
- **NFR1:** The system shall retain submission logs for a minimum of 90 days post-election, accessible to electoral authorities for dispute investigation.

Dispute Resolution

- **FR1:** The system shall provide a third-party auditor interface through which any dispute between a voter and the election authority can be adjudicated using cryptographic proofs.
- **FR2:** The system shall automatically flag and log any ballot confirmation check that returns an invalid proof, making the record available to a neutral arbitrator.
- **NFR1:** Dispute resolution proofs shall be computationally verifiable by a third party within 60 seconds on standard consumer hardware.

Verifiability (Cast-as-Intended)

- **FR1:** After ballot marking but before final submission, the system shall display a human-readable summary of the voter's selections for explicit confirmation.
- **NFR1:** The cast-as-intended confirmation step shall be presented in a format accessible to voters with visual impairments, in compliance with WCAG 2.1 AA.

Verifiability (Recorded-as-Cast)

- **FR1:** The system shall allow each voter to query the bulletin board using their tracking code to confirm that their recorded ballot matches what they submitted.
- **NFR1:** The bulletin board query interface shall be available for at least 14 days following the close of the election.

Verifiability (Counted-as-Recorded)

- **FR1:** The system shall publish the complete set of recorded ballots and the final tally in a form that allows any member of the public to independently recompute and confirm the election result.
- **NFR1:** The published proof and tally data shall be made available in a machine-readable open format (e.g. JSON) within 24 hours of polls closing.

Verifiability (Other)

- **FR1:** The system shall ensure that the list of participating voters is publicly verifiable, allowing any observer to confirm the total number of votes cast without revealing the identity of individual voters.
- **NFR1:** All verifiability artifacts shall be archived and publicly accessible for a minimum of five years following the election.

Everlasting Privacy

- **FR1:** The system shall encrypt ballots using information-theoretically secure methods (e.g. secret sharing) that do not rely on the assumed hardness of any computational problem.
- **NFR1:** Ballot confidentiality shall remain guaranteed even in the event that all cryptographic primitives used in the system are broken at a future date.

Receipt-Freeness

- **FR1:** The system shall be designed such that a voter has no means of demonstrating to a third party which candidate or option they voted for after the vote has been cast.
- **FR2:** In polling station deployments, the system shall discard all intermediate ballot state after the vote is cast, leaving no exportable record of individual choices.
- **NFR1:** The system shall be designed such that any purported vote receipt produced by a voter is cryptographically indistinguishable from a forged one.

Coercion resistance

- **FR1:** The system shall ensure that regardless of the strategy employed by a coercer, a voter retains the ability to cast their intended vote in a way that cannot be detected or verified by the coercer.
- **NFR1:** The coercion-resistance mechanism shall not introduce any observable difference in system behavior or response time compared to normal voting.

Availability

- **FR1:** The system shall be deployed across geographically distributed servers such that the failure of any single node does not interrupt the voting service.
- **FR2:** The system shall implement rate limiting and traffic filtering to mitigate selective denial-of-service attacks targeting individual voter demographics.
- **NFR1:** The system shall maintain 99.9% uptime during the election window, with a maximum tolerated unplanned outage of 5 minutes.

Usability (Efficiency)

- **FR1:** The voting interface shall minimize the number of interactions required to complete ballot submission, removing any steps not strictly necessary to securely cast a vote.

- **NFR1:** The system shall not impose unnecessary latency between voter actions, ensuring that interface responsiveness does not become a barrier to timely vote completion.

Usability (Effectiveness)

- **FR1**: The system shall implement a ballot confirmation screen that prevents accidental submission of incomplete or unintended ballots.
- **NFR1**: Usability testing shall demonstrate that at least 95% of test participants are able to cast a correct ballot without assistance.

Usability (Satisfaction)

- **FR1:** The system shall provide voters with sufficient feedback at each stage of the voting process so that they feel informed and confident that their vote has been correctly received.
- **NFR1:** The system shall be designed to avoid voter confusion or frustration, at the submission and confirmation stages.

Usability (Transparency)

- **FR1:** The system shall publish plain-language documentation explaining how votes are recorded, encrypted, and tallied, accessible from within the voting interface.
- **NFR1:** All published transparency documentation shall be reviewed and approved by an independent electoral observer body before each election.

Usability (Other)

- **FR1:** The system shall provide an accessible voting mode compliant with assistive technologies (screen readers, switch access) for voters with disabilities.
- **NFR1:** Accessibility compliance shall be verified against WCAG 2.1 Level AA prior to each election deployment.

Auditability

- **FR1:** The system shall maintain a tamper-evident record of all significant system events throughout the election, sufficient to allow authorized auditors to reconstruct and verify the conduct of the election after the fact.
- **FR2:** Authorized auditors shall be able to export the full audit log in a standardized format for independent review.
- **NFR1:** The audit log shall be cryptographically signed at regular intervals not exceeding 10 minutes throughout the election period.

Correctness

- **FR1:** The tallying module shall apply a verifiable homomorphic computation or mix-net process to ensure all valid ballots are counted exactly once and no invalid ballots are included.
- **NFR1:** The system shall produce a correctness proof verifiable by an independent auditor within 2 hours of the polls closing.

Privacy

- **FR1:** The system shall encrypt all ballots at the point of entry using public-key encryption, such that no single authority can decrypt an individual ballot.
- **FR2:** Voter identity and ballot content shall be stored in separate, access-controlled databases with no way to query a join between them.
- **NFR1:** Privacy protections shall remain effective against a passive adversary observing all network traffic throughout and after the election.

Unreuseability

- **FR1:** The system shall mark each voter's credential as used immediately upon ballot submission, preventing any subsequent submission attempt using the same credential.
- **NFR1:** The system shall ensure that once a voter's credential has been used to cast a vote, it is permanently invalidated for the remainder of the election period, regardless of the channel or device through which a subsequent submission attempt is made.

Eligibility

- **FR1:** The system shall verify each voter's eligibility against the official electoral register before granting access to the ballot.
- **FR2:** In hybrid deployments, eligibility checks performed at physical polling stations shall be synchronized in real time with the central electoral register to prevent duplicate participation across channels.
- **NFR1:** The system shall complete eligibility verification within a response time that does not meaningfully delay the voter's access to the ballot, even when a large number of voters are authenticating simultaneously.

Robustness

- **FR1:** The system shall continue to accept and process votes correctly if up to one third of back-end nodes fail or behave maliciously.
- **NFR1:** Recovery from a partial system failure shall be automatic and shall not require manual intervention, restoring full service within 2 minutes.

Fairness

- **FR1:** The system shall ensure that no election authority or other party is able to determine any aspect of the election outcome before the voting period has officially ended.
- **NFR1:** The system shall be structured such that finalizing the tally requires the cooperation of multiple independent parties, preventing any single actor from unilaterally accessing results ahead of schedule.

Practicality

- **FR1:** The system shall be operable without requiring voters to install dedicated software, relying only on a standard web browser and internet connection.
- **FR2:** The cryptographic protocols used shall be configurable to scale from small municipal elections (hundreds of voters) to national elections (millions of voters) without architectural changes.
- **NFR1:** The system shall function within the network and hardware constraints typical of rural or low-bandwidth polling environments.

Safe Aggregation

- **FR1:** In hybrid elections, the system shall provide a dedicated aggregation module that accepts both electronically cast votes and scanned paper ballots, combining them into a single verifiable tally.
- **NFR1:** The aggregated tally shall be cryptographically demonstrable as the correct combination of both channels, verifiable by an independent auditor.

Authentication

- **FR1:** The system shall authenticate each voter using multi-factor authentication before granting access to the ballot.
- **NFR1:** Authentication credentials shall be invalidated immediately after use and shall not be reusable within the same election period.

Integrity

- **FR1:** All data transmitted between the voter's device and the voting server shall be protected by end-to-end encryption, preventing any intermediary from altering ballot content in transit.
- **NFR1:** Any detected tampering with stored ballot data shall trigger an automatic system halt.

Forward Secrecy

- **FR1:** The system shall ensure that the confidentiality of previously cast ballots is not compromised if cryptographic keys used during the election are exposed or stolen at a later point in time.
- **NFR1:** Session keys shall be rotated and securely deleted at the end of every individual voting session, with no recoverable copy retained.

Non-Valid Voting Capability

- **FR1:** The system shall allow voters to deliberately submit a blank or intentionally invalid ballot as a legitimate act, without this affecting the validity of other votes or the integrity of the tally.
- **NFR1:** Invalid votes shall be recorded and included in published election statistics as a separate category, independently of the valid vote tally.

Anonymity

- **FR1:** The system shall ensure that the link between a voter's identity and their ballot is permanently and irrevocably broken before the ballot is included in the tally.
- **NFR1:** No combination of system logs, network captures, or database records shall be sufficient to link a specific unsealed ballot to a specific voter.

Revocation

- **FR1:** In internet voting deployments, the system shall allow a voter to cancel and recast their vote at any point before the close of polls, with only the most recent valid submission counted.
- **NFR1:** Vote revocation shall be processed within 5 seconds, and the previous ballot shall be cryptographically invalidated with no possibility of recovery.